

```
// ===== org/foodwise/campus/FoodwiseApplication.java =====
package org.foodwise.campus;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class FoodwiseApplication {
    public static void main(String[] args) {
        SpringApplication.run(FoodwiseApplication.class, args);
    }
}

// ===== org/foodwise/campus/config/SecurityConfig.java =====
package org.foodwise.campus.config;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.autoconfigure.condition.ConditionalOnProperty;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@ConditionalOnProperty(name = "foodwise.security.enabled", havingValue = "true")
public class SecurityConfig {
    @Bean
    PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    UserDetailsService userDetailsService(
        PasswordEncoder passwordEncoder,
        @Value("${foodwise.security.admin-user:foodwise-admin}") String username,
        @Value("${foodwise.security.admin-password:}") String password) {
        if (password == null || password.length() < 10) {
            throw new IllegalStateException("启用登录保护时，FOODWISE_ADMIN_PASSWORD至少需要10位");
        }
        return new InMemoryUserDetailsManager(User.withUsername(username)
            .password(passwordEncoder.encode(password))
            .roles("ADMIN")
            .build());
    }

    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        return http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/login", "/css/**", "/images/**").permitAll()
                .anyRequest().authenticated())
            .csrf(csrf -> csrf.ignoringRequestMatchers("/api/**"))
            .formLogin(form -> form.loginPage("/login").defaultSuccessUrl("/dashboard", true).permitAll())
            .logout(logout -> logout.logoutSuccessUrl("/login?logout").permitAll());
    }
}
```

```

        .build();
    }
}

// ===== org/foodwise/campus/config/LocalSecurityConfig.java =====
package org.foodwise.campus.config;
import org.springframework.boot.autoconfigure.condition.ConditionalOnProperty;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;
@Configuration
@ConditionalOnProperty(name = "foodwise.security.enabled", havingValue = "false", matchIfMissing = true)
public class LocalSecurityConfig {
    @Bean
    SecurityFilterChain localSecurityFilterChain(HttpSecurity http) throws Exception {
        return http
            .authorizeHttpRequests(auth -> auth.anyRequest().permitAll())
            .csrf(csrf -> csrf.disable())
            .build();
    }
}

// ===== org/foodwise/campus/controller/ApiController.java =====
package org.foodwise.campus.controller;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.DecimalMin;
import org.foodwise.campus.model.PredictionResult;
import org.foodwise.campus.model.IntelligentAdvice;
import org.foodwise.campus.model.OfferRecommendation;
import org.foodwise.campus.service.AnalyticsService;
import org.foodwise.campus.service.FoodwiseService;
import org.foodwise.campus.service.PredictionService;
import org.foodwise.campus.service.EnhancedPredictionService;
import org.foodwise.campus.service.DualPredictionService;
import org.foodwise.campus.service.ModelCrossCheckService;
import org.foodwise.campus.service.IntelligentDecisionService;
import org.foodwise.campus.service.OperationFeedbackService;
import org.springframework.http.ResponseEntity;
import org.springframework.http.HttpHeaders;
import org.springframework.http.MediaType;
import org.springframework.validation.annotation.Validated;
import org.springframework.web.bind.annotation.*;
import java.util.LinkedHashMap;
import java.util.Map;
import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.LocalTime;
@Validated
@RestController
@RequestMapping("/api")
public class ApiController {

```

```

private final PredictionService predictionService;
private final FoodwiseService foodwiseService;
private final AnalyticsService analyticsService;
private final IntelligentDecisionService intelligentDecisionService;
private final OperationFeedbackService operationFeedbackService;
private final EnhancedPredictionService enhancedPredictionService;
private final DualPredictionService dualPredictionService;
private final ModelCrossCheckService modelCrossCheckService;
public ApiController(PredictionService predictionService, FoodwiseService foodwiseService,
                    AnalyticsService analyticsService, IntelligentDecisionService intelligentDecisionService,
                    OperationFeedbackService operationFeedbackService,
                    EnhancedPredictionService enhancedPredictionService,
                    DualPredictionService dualPredictionService,
                    ModelCrossCheckService modelCrossCheckService) {
    this.predictionService = predictionService;
    this.foodwiseService = foodwiseService;
    this.analyticsService = analyticsService;
    this.intelligentDecisionService = intelligentDecisionService;
    this.operationFeedbackService = operationFeedbackService;
    this.enhancedPredictionService = enhancedPredictionService;
    this.dualPredictionService = dualPredictionService;
    this.modelCrossCheckService = modelCrossCheckService;
}
@PostMapping("/predictions/run")
public PredictionResult predict(@RequestBody @Validated PredictionRequest request) {
    return predictionService.predict(request.dishId(), request.weather(), request.examWeek(), request.campusEvent)
}
@PostMapping("/predictions/enhanced")
public PredictionResult predictEnhanced(@RequestBody @Validated PredictionRequest request) {
    return enhancedPredictionService.predictEnhanced(
        request.dishId(), request.weather(), request.examWeek(), request.campusEvent());
}
@PostMapping("/predictions/dual")
public Map<String, Object> predictDual(@RequestBody @Validated PredictionRequest request) {
    return dualPredictionService.predictDual(
        request.dishId(), request.weather(), request.examWeek(), request.campusEvent());
}
@PostMapping("/model/cross-check")
public Map<String, Object> crossCheck(@RequestBody @Validated PredictionRequest request) {
    return modelCrossCheckService.crossCheck(
        request.dishId(), request.weather(), request.examWeek(), request.campusEvent());
}
@GetMapping("/model/campus-series")
public Map<String, Object> campusSeries() {
    return analyticsService.campusSeries();
}
@GetMapping("/model/feature-importance")
public java.util.List<Map<String, Object>> featureImportance() {
    return analyticsService.featureImportance();
}

```

```

@PostMapping("/predictions/cold-start")
public PredictionResult predictColdStart(@RequestBody @Validated ColdStartRequest request) {
    return enhancedPredictionService.predictColdStart(
        request.categoryId(), request.weather(), request.examWeek(), request.campusEvent());
}

@PostMapping("/orders/{id}/verify")
public ResponseEntity<Map<String, Object>> verify(@PathVariable @Min(1) long id) {
    foodwiseService.verify(id);
    return ResponseEntity.ok(Map.of("success", true, "message", "核销成功"));
}

@GetMapping("/data/metadata")
public Map<String, Object> metadata() {
    return analyticsService.metadata();
}

@GetMapping("/model/backtest")
public Map<String, Object> backtest() {
    return analyticsService.backtest();
}

@GetMapping("/analytics/financial")
public Map<String, Object> financial() {
    return analyticsService.financial();
}

@GetMapping("/analytics/dashboard")
public Map<String, Object> dashboardAnalytics() {
    return analyticsService.dashboardAnalytics();
}

@GetMapping("/intelligence/dashboard")
public IntelligentAdvice dashboardAdvice() {
    return intelligentDecisionService.dashboardAdvice();
}

@GetMapping("/intelligence/report")
public IntelligentAdvice reportAdvice() {
    return intelligentDecisionService.reportAdvice();
}

@PostMapping("/intelligence/offers/recommend")
public OfferRecommendation recommendOffer(@RequestBody @Validated OfferRequest request) {
    return intelligentDecisionService.recommendOffer(request.dishId());
}

@PostMapping("/offers")
public Map<String, Object> createOffer(@RequestBody @Validated OfferCreateRequest request) {
    long id = foodwiseService.createOffer(request.dishId(), request.quantity(), request.offerPrice(),
        request.startTime(), request.endTime(), request.safetyConfirmed());
    return Map.of("success", true, "id", id, "message", "优惠活动已发布");
}

@PostMapping("/intelligence/advice/{id}/feedback")
public Map<String, Object> adviceFeedback(@PathVariable @Min(1) long id,
    @RequestBody AdviceFeedbackRequest request) {
    intelligentDecisionService.saveFeedback(id, request.adopted(), request.adjustedValue(), request.rejectReason);
    return Map.of("success", true, "message", "经营反馈已记录");
}

```

```

@PostMapping("/operations/feedback")
public Object operationFeedback(@RequestBody @Validated OperationFeedbackRequest request) {
    return operationFeedbackService.submit(request.businessDate(), request.dishId(), request.plannedQty(),
        request.preparedQty(), request.soldQty(), request.discountSoldQty(), request.leftoverQty(),
        request.revenue(), request.weather(), request.eventTag(), request.recommendationAdopted(),
        request.safetyConfirmed(), request.operatorName(), request.note());
}

@GetMapping("/operations/learning-status")
public Map<String, Object> learningStatus() {
    return operationFeedbackService.learningStatus();
}

@PostMapping("/alerts/{id}/resolve")
public Map<String, Object> resolveAlert(@PathVariable @Min(1) long id) {
    foodwiseService.resolveAlert(id);
    return Map.of("success", true, "message", "预警已处理");
}

@GetMapping(value = "/operations/export", produces = "text/csv;charset=UTF-8")
public ResponseEntity<byte[]> exportOperations() {
    return ResponseEntity.ok()
        .header(HttpHeaders.CONTENT_DISPOSITION, "attachment; filename=foodwise-operations.csv")
        .contentType(new MediaType("text", "csv", java.nio.charset.StandardCharsets.UTF_8))
        .body(foodwiseService.exportOperationsCsv());
}

@ExceptionHandler({IllegalArgumentException.class, IllegalStateException.class})
public ResponseEntity<Map<String, Object>> businessError(RuntimeException exception) {
    Map<String, Object> body = new LinkedHashMap<>();
    body.put("success", false);
    body.put("message", exception.getMessage());
    return ResponseEntity.badRequest().body(body);
}

public record PredictionRequest(
    @Min(1) long dishId,
    @NotBlank String weather,
    boolean examWeek,
    boolean campusEvent
) {
}

public record OfferRequest(@Min(1) long dishId) {
}

public record OfferCreateRequest(
    @Min(1) long dishId,
    @Min(1) int quantity,
    @NotNull @DecimalMin("0.01") BigDecimal offerPrice,
    @NotNull LocalTime startTime,
    @NotNull LocalTime endTime,
    boolean safetyConfirmed
) {
}

public record AdviceFeedbackRequest(boolean adopted, String adjustedValue, String rejectReason) {
}

```

```

public record ColdStartRequest(
    @Min(0) long categoryId,
    @NotBlank String weather,
    boolean examWeek,
    boolean campusEvent
) {
}

public record OperationFeedbackRequest(
    @NotNull LocalDate businessDate,
    @Min(1) long dishId,
    @Min(0) int plannedQty,
    @Min(0) int preparedQty,
    @Min(0) int soldQty,
    @Min(0) int discountSoldQty,
    @Min(0) int leftoverQty,
    @NotNull @DecimalMin("0.0") BigDecimal revenue,
    @NotBlank String weather,
    String eventTag,
    boolean recommendationAdopted,
    boolean safetyConfirmed,
    @NotBlank String operatorName,
    String note
) {
}
}

// ===== org/foodwise/campus/controller/PageController.java =====
package org.foodwise.campus.controller;
import org.foodwise.campus.service.FoodwiseService;
import org.foodwise.campus.service.AnalyticsService;
import org.foodwise.campus.service.OperationFeedbackService;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
@Controller
public class PageController {
    private final FoodwiseService foodwiseService;
    private final AnalyticsService analyticsService;
    private final OperationFeedbackService operationFeedbackService;
    public PageController(FoodwiseService foodwiseService, AnalyticsService analyticsService,
        OperationFeedbackService operationFeedbackService) {
        this.foodwiseService = foodwiseService;
        this.analyticsService = analyticsService;
        this.operationFeedbackService = operationFeedbackService;
    }
    @ModelAttribute
    public void commonModel(Model model) {
        model.addAttribute("pendingAlertCount", foodwiseService.pendingAlertCount());
    }
    @GetMapping({"/", "/dashboard"})
    public String dashboard(Model model) {

```

```

        model.addAttribute("active", "dashboard");
        model.addAttribute("pageTitle", "经营驾驶舱");
        model.addAllAttributes(foodwiseService.dashboard());
        addAnalytics(model);
        return "dashboard";
    }

    @GetMapping("/login")
    public String login(Model model) {
        model.addAttribute("pageTitle", "运营登录");
        return "login";
    }

    @GetMapping("/stalls")
    public String stalls(Model model) {
        model.addAttribute("active", "stalls");
        model.addAttribute("pageTitle", "档口与菜品");
        model.addAttribute("stalls", foodwiseService.stalls());
        model.addAttribute("dishes", foodwiseService.dishes());
        addAnalytics(model);
        return "stalls";
    }

    @GetMapping("/prediction")
    public String prediction(Model model) {
        model.addAttribute("active", "prediction");
        model.addAttribute("pageTitle", "备餐决策中心");
        model.addAttribute("dishes", foodwiseService.dishes());
        addAnalytics(model);
        return "prediction";
    }

    @GetMapping("/offers")
    public String offers(Model model) {
        model.addAttribute("active", "offers");
        model.addAttribute("pageTitle", "限时优惠");
        model.addAttribute("offers", foodwiseService.offers());
        model.addAttribute("dishes", foodwiseService.dishes());
        addAnalytics(model);
        return "offers";
    }

    @GetMapping("/orders")
    public String orders(Model model) {
        model.addAttribute("active", "orders");
        model.addAttribute("pageTitle", "订单核销");
        model.addAttribute("orders", foodwiseService.orders());
        addAnalytics(model);
        return "orders";
    }

    @GetMapping("/operations/feedback")
    public String operationFeedback(Model model) {
        model.addAttribute("active", "operation-feedback");
        model.addAttribute("pageTitle", "经营数据回传");
        model.addAttribute("dishes", foodwiseService.dishes());
    }

```

```

        model.addAttribute("learningStatus", operationFeedbackService.learningStatus());
        model.addAttribute("recentFeedback", operationFeedbackService.recentFeedback());
        addAnalytics(model);
        return "operation-feedback";
    }

    @GetMapping("/reports")
    public String reports(Model model) {
        model.addAttribute("active", "reports");
        model.addAttribute("pageTitle", "经营复盘报告");
        model.addAllAttributes(foodwiseService.report());
        addAnalytics(model);
        return "reports";
    }

    @GetMapping("/alerts")
    public String alerts(Model model) {
        model.addAttribute("active", "alerts");
        model.addAttribute("pageTitle", "经营预警中心");
        model.addAttribute("alerts", foodwiseService.alerts());
        addAnalytics(model);
        return "alerts";
    }

    @GetMapping("/insights/realtime")
    public String realtimeInsights(Model model) {
        model.addAttribute("active", "insights-realtime");
        model.addAttribute("pageTitle", "实时经营分析");
        model.addAllAttributes(foodwiseService.dashboard());
        addAnalytics(model);
        return "insights-realtime";
    }

    @GetMapping("/insights/demand")
    public String demandInsights(Model model) {
        model.addAttribute("active", "insights-demand");
        model.addAttribute("pageTitle", "需求预测分析");
        model.addAllAttributes(foodwiseService.dashboard());
        addAnalytics(model);
        return "insights-demand";
    }

    @GetMapping("/insights/waste")
    public String wasteInsights(Model model) {
        model.addAttribute("active", "insights-waste");
        model.addAttribute("pageTitle", "减损场景分析");
        model.addAllAttributes(foodwiseService.dashboard());
        addAnalytics(model);
        return "insights-waste";
    }

    @GetMapping("/about")
    public String about(Model model) {
        model.addAttribute("active", "about");
        model.addAttribute("pageTitle", "服务说明");
        addAnalytics(model);
    }

```

```

        return "about";
    }

    private void addAnalytics(Model model) {
        model.addAttribute("analytics", analyticsService.dashboardAnalytics());
    }
}

// ===== org/foodwise/campus/model/IntelligentAdvice.java =====
package org.foodwise.campus.model;
import java.util.List;
public record IntelligentAdvice(
    long id,
    String title,
    String summary,
    String riskLevel,
    List<String> reasons,
    List<Action> actions,
    List<String> warnings,
    String source,
    boolean aiGenerated,
    String generatedAt
) {
    public record Action(String type, String title, String detail, String priority) {
    }
}

// ===== org/foodwise/campus/model/OfferRecommendation.java =====
package org.foodwise.campus.model;
import java.math.BigDecimal;
public record OfferRecommendation(
    long dishId,
    String dishName,
    int recommendedQty,
    BigDecimal recommendedPrice,
    int discountRate,
    String startTime,
    String endTime,
    int expectedRecoveredQty,
    BigDecimal estimatedRevenue,
    String riskLevel,
    IntelligentAdvice advice
) {
}

// ===== org/foodwise/campus/model/OperationLearningResult.java =====
package org.foodwise.campus.model;
import java.math.BigDecimal;
import java.time.LocalDate;
public record OperationLearningResult(
    long operationId,
    long dishId,
    String dishName,
    LocalDate businessDate,
    boolean updated,
    int sampleCount,
    BigDecimal baselineBefore,

```

```

        BigDecimal baselineAfter,
        BigDecimal adjustmentRate,
        BigDecimal leftoverRate,
        String learningMethod,
        String summary,
        String nextAction
    ) {
    }

    // ===== org/foodwise/campus/model/PredictionResult.java =====
    package org.foodwise.campus.model;
    import java.math.BigDecimal;
    import java.util.List;
    public record PredictionResult(
        long dishId,
        String dishName,
        int predictedLow,
        int predictedMid,
        int predictedHigh,
        int firstBatch,
        int replenishQty,
        int stopHour,
        int stopMinute,
        BigDecimal confidence,
        String soldOutRisk,
        String leftoverRisk,
        List<Factor> factors,
        IntelligentAdvice advice
    ) {
        public record Factor(String name, String detail, BigDecimal value, String type) {
        }
    }

    // ===== org/foodwise/campus/repository/FoodwiseRepository.java =====
    package org.foodwise.campus.repository;
    import org.springframework.jdbc.core.JdbcTemplate;
    import org.springframework.jdbc.support.GeneratedKeyHolder;
    import org.springframework.jdbc.support.KeyHolder;
    import org.springframework.stereotype.Repository;
    import java.sql.PreparedStatement;
    import java.math.BigDecimal;
    import java.math.RoundingMode;
    import java.time.LocalDate;
    import java.time.LocalDateTime;
    import java.util.List;
    import java.util.Map;
    @Repository
    public class FoodwiseRepository {
        private final JdbcTemplate jdbcTemplate;
        public FoodwiseRepository(JdbcTemplate jdbcTemplate) {
            this.jdbcTemplate = jdbcTemplate;
        }
        public Map<String, Object> dashboardSummary() {
            return jdbcTemplate.queryForMap("""

```

```

SELECT COALESCE(SUM(prepared_qty), 0) AS prepared,
       COALESCE(SUM(sold_qty), 0) AS sold,
       COALESCE(SUM(leftover_qty), 0) AS leftover,
       COALESCE(SUM(discount_sold_qty), 0) AS rescued,
       COALESCE(SUM(revenue), 0) AS revenue,
       COALESCE(AVG(CASE WHEN prepared_qty = 0 THEN 0
                       ELSE (1 - ABS(planned_qty - sold_qty) * 1.0 / prepared_qty) * 100 END), 0) AS accuracy
FROM daily_operation
WHERE business_date = (SELECT MAX(business_date) FROM daily_operation)
""");
}

public List<Map<String, Object>> trend() {
    return jdbcTemplate.queryForList("""
        SELECT business_date AS business_day,
               SUM(prepared_qty) AS prepared,
               SUM(sold_qty) AS sold,
               SUM(leftover_qty) AS leftover,
               SUM(discount_sold_qty) AS rescued
        FROM daily_operation
        GROUP BY business_date
        ORDER BY business_date
        """);
}

public List<Map<String, Object>> stalls() {
    return jdbcTemplate.queryForList("""
        SELECT s.id, s.name, s.category, s.location, s.manager_name, s.status, s.rating,
               COUNT(DISTINCT d.id) AS dish_count,
               COALESCE(SUM(CASE WHEN o.business_date = (SELECT MAX(business_date) FROM daily_operation) THEN
                           COALESCE(SUM(CASE WHEN o.business_date = (SELECT MAX(business_date) FROM daily_operation) THEN
                               FROM stall s
                               LEFT JOIN dish d ON d.stall_id = s.id
                               LEFT JOIN daily_operation o ON o.dish_id = d.id
                               GROUP BY s.id, s.name, s.category, s.location, s.manager_name, s.status, s.rating
                               ORDER BY s.id
                               """);
        """);
}

public List<Map<String, Object>> dishes() {
    return jdbcTemplate.queryForList("""
        SELECT d.id, d.name, d.category, d.price, d.unit_cost, d.prep_minutes,
               d.active, d.color, s.name AS stall_name,
               COALESCE(AVG(o.sold_qty), 0) AS avg_sales,
               COALESCE(AVG(CASE WHEN o.prepared_qty = 0 THEN 0
                               ELSE o.leftover_qty * 100.0 / o.prepared_qty END), 0) AS leftover_rate
        FROM dish d
        JOIN stall s ON s.id = d.stall_id
        LEFT JOIN daily_operation o ON o.dish_id = d.id
        GROUP BY d.id, d.name, d.category, d.price, d.unit_cost, d.prep_minutes,
               d.active, d.color, s.name
        ORDER BY d.id
        """);
}

```

```

}

public Map<String, Object> dish(long dishId) {
    return jdbcTemplate.queryForMap("""
        SELECT d.*, s.name AS stall_name
        FROM dish d JOIN stall s ON s.id = d.stall_id
        WHERE d.id = ?
        """, dishId);
}

public List<Integer> recentSales(long dishId) {
    return jdbcTemplate.queryForList("""
        SELECT sold_qty FROM daily_operation
        WHERE dish_id = ? ORDER BY business_date DESC LIMIT 7
        """, Integer.class, dishId);
}

public Map<String, Object> latestDishOperation(long dishId) {
    List<Map<String, Object>> rows = jdbcTemplate.queryForList("""
        SELECT o.business_date, o.prepared_qty, o.sold_qty, o.leftover_qty,
            o.discount_sold_qty, o.weather, o.event_tag
        FROM daily_operation o
        WHERE o.dish_id = ?
        ORDER BY o.business_date DESC
        LIMIT 1
        """, dishId);
    if (rows.isEmpty()) {
        throw new IllegalArgumentException("该菜品暂无经营记录，无法生成优惠建议");
    }
    return rows.get(0);
}

public List<Map<String, Object>> riskDishes() {
    return jdbcTemplate.queryForList("""
        SELECT d.id, d.name, s.name AS stall_name, o.prepared_qty, o.sold_qty, o.leftover_qty,
            CASE
                WHEN o.leftover_qty >= 7 THEN '高'
                WHEN o.leftover_qty >= 4 THEN '中'
                ELSE '低'
            END AS risk_level
        FROM daily_operation o
        JOIN dish d ON d.id = o.dish_id
        JOIN stall s ON s.id = d.stall_id
        WHERE o.business_date = (SELECT MAX(business_date) FROM daily_operation)
        ORDER BY o.leftover_qty DESC
        LIMIT 6
        """);
}

public List<Map<String, Object>> offers() {
    return jdbcTemplate.queryForList("""
        SELECT f.id, f.title, f.original_price, f.offer_price, f.total_qty, f.remaining_qty,
            f.start_time, f.end_time, f.pickup_location, f.status,
            d.name AS dish_name, s.name AS stall_name,
            ROUND((1 - f.offer_price / f.original_price) * 100) AS discount_rate
    """);
}

```

```

        FROM discount_offer f
        JOIN dish d ON d.id = f.dish_id
        JOIN stall s ON s.id = d.stall_id
        ORDER BY f.id DESC
        """);
    }

    public long createOffer(long dishId, String title, BigDecimal originalPrice, BigDecimal offerPrice,
        int totalQty, LocalDateTime startTime, LocalDateTime endTime,
        String pickupLocation) {
        KeyHolder keyHolder = new GeneratedKeyHolder();
        jdbcTemplate.update(connection -> {
            PreparedStatement statement = connection.prepareStatement("""
                INSERT INTO discount_offer(dish_id, title, original_price, offer_price, total_qty,
                    remaining_qty, start_time, end_time, pickup_location, status)
                VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, '进行中')
                """, new String[]{"id"});
            statement.setLong(1, dishId);
            statement.setString(2, title);
            statement.setBigDecimal(3, originalPrice);
            statement.setBigDecimal(4, offerPrice);
            statement.setInt(5, totalQty);
            statement.setInt(6, totalQty);
            statement.setObject(7, startTime);
            statement.setObject(8, endTime);
            statement.setString(9, pickupLocation);
            return statement;
        }, keyHolder);
        Number key = keyHolder.getKey();
        if (key == null) throw new IllegalStateException("优惠活动创建失败");
        return key.longValue();
    }

    public List<Map<String, Object>> orders() {
        return jdbcTemplate.queryForList("""
            SELECT o.id, o.order_no, o.student_alias, o.quantity, o.amount, o.pickup_code,
                o.status, o.created_at, o.verified_at, f.title, d.name AS dish_name,
                f.pickup_location
            FROM meal_order o
            JOIN discount_offer f ON f.id = o.offer_id
            JOIN dish d ON d.id = f.dish_id
            ORDER BY o.created_at DESC, o.id DESC
            """);
    }

    public Map<String, Object> reportSummary() {
        return jdbcTemplate.queryForMap("""
            SELECT ROUND(SUM(CASE WHEN o.phase = '基线期' THEN o.leftover_qty ELSE 0 END) * 100.0 /
                NULLIF(SUM(CASE WHEN o.phase = '基线期' THEN o.prepared_qty ELSE 0 END), 0), 1) AS baselin
            ROUND(SUM(CASE WHEN o.phase = '干预期' THEN o.leftover_qty ELSE 0 END) * 100.0 /
                NULLIF(SUM(CASE WHEN o.phase = '干预期' THEN o.prepared_qty ELSE 0 END), 0), 1) AS interve
            ROUND(SUM(CASE WHEN o.phase = '基线期' THEN o.sold_qty ELSE 0 END) * 100.0 /
                NULLIF(SUM(CASE WHEN o.phase = '基线期' THEN o.prepared_qty ELSE 0 END), 0), 1) AS baselin
        """);
    }

```

```

ROUND(SUM(CASE WHEN o.phase = '干预期' THEN o.sold_qty ELSE 0 END) * 100.0 /
    NULLIF(SUM(CASE WHEN o.phase = '干预期' THEN o.prepared_qty ELSE 0 END), 0), 1) AS interve
ROUND(SUM(CASE WHEN o.phase = '基线期' THEN o.revenue ELSE 0 END), 2) AS baseline_revenue,
ROUND(SUM(CASE WHEN o.phase = '干预期' THEN o.revenue ELSE 0 END), 2) AS intervention_revenue,
SUM(CASE WHEN o.phase = '干预期' THEN o.discount_sold_qty ELSE 0 END) AS rescued_meals,
ROUND(SUM(CASE WHEN o.phase = '基线期' THEN o.leftover_qty * d.unit_cost ELSE 0 END) -
    SUM(CASE WHEN o.phase = '干预期' THEN o.leftover_qty * d.unit_cost ELSE 0 END), 2) AS a
ROUND(AVG(CASE WHEN o.phase = '基线期' AND o.sold_qty > 0
    THEN ABS(o.planned_qty - o.sold_qty) * 100.0 / o.sold_qty END), 1) AS baseline_prediction_er
ROUND(AVG(CASE WHEN o.phase = '干预期' AND o.sold_qty > 0
    THEN ABS(o.planned_qty - o.sold_qty) * 100.0 / o.sold_qty END), 1) AS prediction_error,
ROUND(AVG(CASE WHEN o.phase = '干预期'
    THEN CASE WHEN o.recommendation_adopted THEN 100 ELSE 0 END END), 1) AS adoption_rate
FROM daily_operation o
JOIN dish d ON d.id = o.dish_id
""");
}

public Map<String, Object> dataMetadataSummary() {
    return jdbcTemplate.queryForMap("""
        SELECT COUNT(*) AS records,
               COUNT(DISTINCT o.dish_id) AS dishes,
               COUNT(DISTINCT d.stall_id) AS stalls,
               COUNT(DISTINCT o.business_date) AS operating_days,
               MIN(o.business_date) AS period_start,
               MAX(o.business_date) AS period_end,
               SUM(CASE WHEN o.sold_qty + o.leftover_qty = o.prepared_qty THEN 0 ELSE 1 END) AS conservation
               SUM(CASE WHEN o.discount_sold_qty <= o.sold_qty THEN 0 ELSE 1 END) AS discount_violations
        FROM daily_operation o
        JOIN dish d ON d.id = o.dish_id
        """);
}

public List<Map<String, Object>> dataSources() {
    return jdbcTemplate.queryForList("""
        SELECT source_id, source_name, source_type, provider, source_url, license_info,
               local_path, project_usage, truth_boundary, updated_at
        FROM data_source_metadata
        ORDER BY source_id
        """);
}

public List<Map<String, Object>> operationRows() {
    return jdbcTemplate.queryForList("""
        SELECT o.business_date, o.phase, o.dish_id, d.name AS dish_name, s.name AS stall_name,
               o.planned_qty, o.prepared_qty, o.sold_qty, o.discount_sold_qty, o.leftover_qty,
               o.revenue, o.weather, o.event_tag, o.recommendation_adopted, d.unit_cost
        FROM daily_operation o
        JOIN dish d ON d.id = o.dish_id
        JOIN stall s ON s.id = d.stall_id
        ORDER BY o.dish_id, o.business_date
        """);
}

```

```

public List<Map<String, Object>> financialByPhase() {
    return jdbcTemplate.queryForList("""
        SELECT o.phase,
               SUM(o.prepared_qty) AS prepared_qty,
               SUM(o.sold_qty) AS sold_qty,
               SUM(o.leftover_qty) AS leftover_qty,
               ROUND(SUM(o.revenue), 2) AS revenue,
               ROUND(SUM(o.prepared_qty * d.unit_cost), 2) AS food_cost,
               ROUND(SUM(o.leftover_qty * d.unit_cost), 2) AS leftover_cost,
               ROUND(SUM(o.revenue) - SUM(o.prepared_qty * d.unit_cost), 2) AS contribution_profit,
               ROUND((SUM(o.revenue) - SUM(o.prepared_qty * d.unit_cost)) * 100.0 /
                     NULLIF(SUM(o.revenue), 0), 1) AS contribution_margin
        FROM daily_operation o
        JOIN dish d ON d.id = o.dish_id
        GROUP BY o.phase
        ORDER BY MIN(o.business_date)
        """);
}

public List<Map<String, Object>> dailyForecastSeries() {
    return jdbcTemplate.queryForList("""
        SELECT business_date AS business_day,
               SUM(planned_qty) AS predicted,
               SUM(sold_qty) AS actual,
               ROUND(SUM(planned_qty) * 0.92, 0) AS predicted_low,
               ROUND(SUM(planned_qty) * 1.08, 0) AS predicted_high
        FROM daily_operation
        GROUP BY business_date
        ORDER BY business_date
        """);
}

public List<Map<String, Object>> latestDishOperations() {
    return jdbcTemplate.queryForList("""
        SELECT d.name, s.name AS stall_name, o.planned_qty, o.prepared_qty, o.sold_qty, o.leftover_qty,
               o.discount_sold_qty,
               ROUND(o.sold_qty * 100.0 / NULLIF(o.prepared_qty, 0), 1) AS sell_through_rate
        FROM daily_operation o
        JOIN dish d ON d.id = o.dish_id
        JOIN stall s ON s.id = o.stall_id
        WHERE o.business_date = (SELECT MAX(business_date) FROM daily_operation)
        ORDER BY d.id
        """);
}

public List<Map<String, Object>> stallEfficiency() {
    return jdbcTemplate.queryForList("""
        SELECT s.name,
               ROUND(SUM(o.sold_qty) * 100.0 / NULLIF(SUM(o.prepared_qty), 0), 1) AS score
        FROM stall s
        JOIN dish d ON d.stall_id = s.id
        JOIN daily_operation o ON o.dish_id = d.id
        GROUP BY s.id, s.name
    """);
}

```

```

        ORDER BY score DESC
        """);
    }

    public OperationSaveResult upsertOperation(LocalDate businessDate, long dishId, int plannedQty,
                                              int preparedQty, int soldQty, int discountSoldQty,
                                              int leftoverQty, BigDecimal revenue, String weather,
                                              String eventTag, boolean recommendationAdopted) {
        List<Long> existing = jdbcTemplate.queryForList("""
            SELECT id FROM daily_operation
            WHERE business_date = ? AND dish_id = ?
            ORDER BY id DESC LIMIT 1
            """, Long.class, businessDate, dishId);

        if (!existing.isEmpty()) {
            long operationId = existing.get(0);
            jdbcTemplate.update("""
                UPDATE daily_operation SET phase='运营回传', planned_qty=?, prepared_qty=?,
                sold_qty=?, discount_sold_qty=?, leftover_qty=?, revenue=?, weather=?,
                event_tag=?, recommendation_adopted=?
                WHERE id=?
                """, plannedQty, preparedQty, soldQty, discountSoldQty, leftoverQty, revenue,
                weather, eventTag, recommendationAdopted, operationId);
            return new OperationSaveResult(operationId, true);
        }

        KeyHolder keyHolder = new GeneratedKeyHolder();
        jdbcTemplate.update(connection -> {
            PreparedStatement statement = connection.prepareStatement("""
                INSERT INTO daily_operation(business_date, phase, dish_id, planned_qty,
                prepared_qty, sold_qty, discount_sold_qty, leftover_qty, revenue,
                weather, event_tag, recommendation_adopted)
                VALUES (?, '运营回传', ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
                """, new String[]{"id"});

            statement.setObject(1, businessDate);
            statement.setLong(2, dishId);
            statement.setInt(3, plannedQty);
            statement.setInt(4, preparedQty);
            statement.setInt(5, soldQty);
            statement.setInt(6, discountSoldQty);
            statement.setInt(7, leftoverQty);
            statement.setBigDecimal(8, revenue);
            statement.setString(9, weather);
            statement.setString(10, eventTag);
            statement.setBoolean(11, recommendationAdopted);
            return statement;
        }, keyHolder);

        Number key = keyHolder.getKey();
        if (key == null) throw new IllegalStateException("经营记录写入失败");
        return new OperationSaveResult(key.longValue(), false);
    }

    public void saveOperationFeedback(long operationId, String operatorName, boolean safetyConfirmed,
                                     String note, String submissionType) {

```

```

jdbcTemplate.update("""
    INSERT INTO operation_feedback_record(operation_id, operator_name, safety_confirmed,
        submission_note, submission_type)
    VALUES (?, ?, ?, ?, ?)
    """, operationId, operatorName, safetyConfirmed, note, submissionType);
}

public void saveLearningLog(long operationId, long dishId, String methodName, int sampleCount,
    BigDecimal baselineBefore, BigDecimal baselineAfter,
    BigDecimal adjustmentRate) {
    jdbcTemplate.update("""
        INSERT INTO model_learning_log(operation_id, dish_id, method_name, sample_count,
            baseline_before, baseline_after, adjustment_rate)
        VALUES (?, ?, ?, ?, ?, ?, ?)
        """, operationId, dishId, methodName, sampleCount, baselineBefore, baselineAfter, adjustmentRate);
}

public Map<String, Object> learningStatus() {
    return jdbcTemplate.queryForMap("""
        SELECT COUNT(*) AS learning_updates,
            COUNT(DISTINCT dish_id) AS learned_dishes,
            COALESCE(MAX(created_at), CURRENT_TIMESTAMP) AS latest_update,
            COALESCE(AVG(ABS(adjustment_rate)), 0) AS average_adjustment
        FROM model_learning_log
        """);
}

public List<Map<String, Object>> recentOperationFeedback() {
    return jdbcTemplate.queryForList("""
        SELECT f.id, f.submission_type, f.operator_name, f.safety_confirmed,
            f.submission_note, f.submitted_at, o.business_date, o.prepared_qty,
            o.sold_qty, o.discount_sold_qty, o.leftover_qty, o.weather,
            d.name AS dish_name, s.name AS stall_name,
            l.baseline_before, l.baseline_after, l.adjustment_rate
        FROM operation_feedback_record f
        JOIN daily_operation o ON o.id = f.operation_id
        JOIN dish d ON d.id = o.dish_id
        JOIN stall s ON s.id = d.stall_id
        LEFT JOIN model_learning_log l ON l.id = (
            SELECT MAX(12.id) FROM model_learning_log 12 WHERE 12.operation_id = o.id
        )
        ORDER BY f.submitted_at DESC, f.id DESC
        LIMIT 12
        """);
}

public void replaceOperationAlerts(LocalDate businessDate, long dishId, int preparedQty,
    int soldQty, int leftoverQty) {
    jdbcTemplate.update("DELETE FROM operation_alert WHERE business_date=? AND dish_id=? AND status='待处理'",
        businessDate, dishId);
    double leftoverRate = preparedQty == 0 ? 0 : leftoverQty * 100.0 / preparedQty;
    if (leftoverQty >= 7 || leftoverRate >= 10) {
        jdbcTemplate.update("""
            INSERT INTO operation_alert(business_date, dish_id, alert_type, risk_level,

```

```

        alert_message, status)
VALUES (?, ?, '剩余偏高', ?, ?, '待处理')
""", businessDate, dishId, leftoverRate >= 15 ? "高" : "中",
"闭餐剩余" + leftoverQty + "份, 剩余率" + BigDecimal.valueOf(leftoverRate).setScale(1, RoundingM
}
double sellThrough = preparedQty == 0 ? 0 : soldQty * 100.0 / preparedQty;
if (preparedQty > 0 && sellThrough >= 98) {
    jdbcTemplate.update("""
        INSERT INTO operation_alert(business_date, dish_id, alert_type, risk_level,
            alert_message, status)
VALUES (?, ?, '售罄风险', '中', ?, '待处理')
""", businessDate, dishId, "售罄率达到" + BigDecimal.valueOf(sellThrough).setScale(1, RoundingMode
    }
}

public List<Map<String, Object>> alerts() {
    return jdbcTemplate.queryForList("""
        SELECT a.id, a.business_date, a.alert_type, a.risk_level, a.alert_message,
            a.status, a.handled_at, a.created_at, d.name AS dish_name,
            s.name AS stall_name
        FROM operation_alert a
        LEFT JOIN dish d ON d.id=a.dish_id
        LEFT JOIN stall s ON s.id=d.stall_id
        ORDER BY CASE a.status WHEN '待处理' THEN 0 ELSE 1 END,
            CASE a.risk_level WHEN '高' THEN 0 WHEN '中' THEN 1 ELSE 2 END,
            a.created_at DESC
        """);
}

public int pendingAlertCount() {
    Integer count = jdbcTemplate.queryForObject("SELECT COUNT(*) FROM operation_alert WHERE status='待处理'", Int
    return count == null ? 0 : count;
}

public int resolveAlert(long alertId) {
    return jdbcTemplate.update("""
        UPDATE operation_alert SET status='已处理', handled_at=CURRENT_TIMESTAMP
        WHERE id=? AND status='待处理'
        """, alertId);
}

public List<Map<String, Object>> operationExportRows() {
    return jdbcTemplate.queryForList("""
        SELECT o.business_date, s.name AS stall_name, d.name AS dish_name, o.phase,
            o.planned_qty, o.prepared_qty, o.sold_qty, o.discount_sold_qty,
            o.leftover_qty, o.revenue, o.weather, o.event_tag,
            o.recommendation_adopted
        FROM daily_operation o
        JOIN dish d ON d.id=o.dish_id
        JOIN stall s ON s.id=d.stall_id
        ORDER BY o.business_date DESC, s.id, d.id
        """);
}

public void savePrediction(long dishId, LocalDate targetDate, int low, int mid, int high,

```

```

        int firstBatch, int replenish, BigDecimal confidence,
        BigDecimal weatherFactor, BigDecimal calendarFactor, BigDecimal trendFactor) {
jdbcTemplate.update("""
    INSERT INTO prediction_record(target_date, dish_id, predicted_low, predicted_mid,
        predicted_high, first_batch, replenish_qty, confidence, weather_factor,
        calendar_factor, trend_factor)
    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
    """, targetDate, dishId, low, mid, high, firstBatch, replenish, confidence,
        weatherFactor, calendarFactor, trendFactor);
}

public long saveAdvice(String scenario, String title, String summary, String riskLevel,
        String source, boolean aiGenerated, String inputSnapshot, String structuredResult) {
    KeyHolder keyHolder = new GeneratedKeyHolder();
    jdbcTemplate.update(connection -> {
        PreparedStatement statement = connection.prepareStatement("""
            INSERT INTO ai_advice_record(scenario, title, summary, risk_level, source,
                ai_generated, input_snapshot, structured_result)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
            """, new String[]{"id"});
        statement.setString(1, scenario);
        statement.setString(2, title);
        statement.setString(3, summary);
        statement.setString(4, riskLevel);
        statement.setString(5, source);
        statement.setBoolean(6, aiGenerated);
        statement.setString(7, inputSnapshot);
        statement.setString(8, structuredResult);
        return statement;
    }, keyHolder);
    Number key = keyHolder.getKey();
    return key == null ? 0 : key.longValue();
}

public void saveAdviceFeedback(long adviceId, boolean adopted, String adjustedValue, String rejectReason) {
    jdbcTemplate.update("""
        INSERT INTO advice_feedback(advice_id, adopted, adjusted_value, reject_reason, operator_name)
        VALUES (?, ?, ?, ?, '档口操作员')
        """, adviceId, adopted, adjustedValue, rejectReason);
}

public int verifyOrder(long orderId, String operator) {
    int updated = jdbcTemplate.update("""
        UPDATE meal_order SET status = '已核销', verified_at = CURRENT_TIMESTAMP
        WHERE id = ? AND status = '待取餐'
        """, orderId);
    if (updated == 1) {
        jdbcTemplate.update("""
            INSERT INTO verification_record(order_id, result, operator_name, remark)
            VALUES (?, '成功', ?, '档口取餐核销')
            """, orderId, operator);
    }
    return updated;
}

```

```

    }

    public long countFeedbackSinceLastUpdate(long dishId) {
        Long count = jdbcTemplate.queryForObject("""
            SELECT COUNT(*) FROM operation_feedback_record f
            JOIN daily_operation o ON o.id = f.operation_id
            WHERE o.dish_id = ?
            AND f.submitted_at > COALESCE((
                SELECT MAX(created_at) FROM model_learning_log
                WHERE dish_id = ?
            ), '2000-01-01')
            """, Long.class, dishId, dishId);

        return count == null ? 0 : count;
    }

    public List<Map<String, Object>> recentOperationsByDish(long dishId) {
        return jdbcTemplate.queryForList("""
            SELECT business_date, weather, sold_qty, prepared_qty
            FROM daily_operation
            WHERE dish_id = ?
            ORDER BY business_date DESC
            LIMIT 5
            """, dishId);
    }

    public record OperationSaveResult(long operationId, boolean updated) {
    }
}

// ===== org/foodwise/campus/service/FoodwiseService.java =====
package org.foodwise.campus.service;

import org.foodwise.campus.repository.FoodwiseRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.nio.charset.StandardCharsets;

@Service
public class FoodwiseService {
    private final FoodwiseRepository repository;

    public FoodwiseService(FoodwiseRepository repository) {
        this.repository = repository;
    }

    public Map<String, Object> dashboard() {
        Map<String, Object> data = new LinkedHashMap<>();
        Map<String, Object> summary = new LinkedHashMap<>(repository.dashboardSummary());
        Map<String, Object> report = repository.reportSummary();
        double avoidedLoss = num(report.get("avoided_loss"));
        long baselineLeftover = 0L;
        long interventionLeftover = 0L;
    }
}

```

```

for (Map<String, Object> row : repository.operationRows()) {
    long leftover = ((Number) row.get("leftover_qty")).longValue();
    String phase = String.valueOf(row.get("phase"));
    if ("基线期".equals(phase)) baselineLeftover += leftover;
    else if ("干预期".equals(phase)) interventionLeftover += leftover;
}
long reducedPieces = Math.max(0L, baselineLeftover - interventionLeftover);
double wasteWeightKg = reducedPieces * 0.3d;
double annualFactor = 260d / 14d;
long costSaved = Math.round(avoidedLoss * annualFactor);
long carbonSaved = Math.round(wasteWeightKg * 1.1d * annualFactor);
long waterSaved = Math.round(wasteWeightKg * 250d * annualFactor);
summary.put("cost_saved", costSaved);
summary.put("carbon_saved", carbonSaved);
summary.put("water_saved", waterSaved);
data.put("summary", summary);
data.put("trend", repository.trend());
data.put("risks", repository.riskDishes());
data.put("offers", repository.offers());
return data;
}

private static double num(Object value) {
    if (value == null) return 0d;
    if (value instanceof Number n) return n.doubleValue();
    try { return Double.parseDouble(String.valueOf(value)); } catch (Exception e) { return 0d; }
}

public List<Map<String, Object>> stalls() {
    return repository.stalls();
}

public List<Map<String, Object>> dishes() {
    return repository.dishes();
}

public List<Map<String, Object>> offers() {
    return repository.offers();
}

public List<Map<String, Object>> orders() {
    return repository.orders();
}

@Transactional
public long createOffer(long dishId, int quantity, BigDecimal offerPrice, LocalTime startTime,
    LocalTime endTime, boolean safetyConfirmed) {
    if (!safetyConfirmed) throw new IllegalArgumentException("发布前必须确认餐品仍处于当餐安全售卖时段");
    if (quantity <= 0) throw new IllegalArgumentException("优惠数量必须大于0");
    if (startTime == null || endTime == null || !endTime.isAfter(startTime)) {
        throw new IllegalArgumentException("优惠截止时间必须晚于开始时间");
    }
    Map<String, Object> dish = repository.dish(dishId);
    Map<String, Object> operation = repository.latestDishOperation(dishId);
    int leftover = ((Number) operation.get("leftover_qty")).intValue();
    if (quantity > leftover) throw new IllegalArgumentException("优惠数量不能超过当前剩余" + leftover + "份");
}

```

```

BigDecimal originalPrice = new BigDecimal(String.valueOf(dish.get("price")));
BigDecimal unitCost = new BigDecimal(String.valueOf(dish.get("unit_cost")));
if (offerPrice == null || offerPrice.compareTo(unitCost) < 0 || offerPrice.compareTo(originalPrice) >= 0) {
    throw new IllegalArgumentException("优惠价格应高于单位成本且低于原价");
}
LocalDate date = LocalDate.now();
String title = String.valueOf(dish.get("name")) + " • 当餐限时优惠";
String pickupLocation = String.valueOf(dish.get("stall_name"));
return repository.createOffer(dishId, title, originalPrice, offerPrice, quantity,
    LocalDateTime.of(date, startTime), LocalDateTime.of(date, endTime), pickupLocation);
}

public List<Map<String, Object>> alerts() {
    return repository.alerts();
}

public int pendingAlertCount() {
    return repository.pendingAlertCount();
}

@Transactional
public void resolveAlert(long alertId) {
    if (repository.resolveAlert(alertId) != 1) {
        throw new IllegalStateException("预警不存在或已经处理");
    }
}

public byte[] exportOperationsCsv() {
    StringBuilder csv = new StringBuilder("\uFEFF营业日期, 档口, 菜品, 数据阶段, 建议备餐, 实际备餐, 售出, 优惠售出, 闭
for (Map<String, Object> row : repository.operationExportRows()) {
    csv.append(csvCell(row.get("business_date"))).append(',')
        .append(csvCell(row.get("stall_name"))).append(',')
        .append(csvCell(row.get("dish_name"))).append(',')
        .append(csvCell(row.get("phase"))).append(',')
        .append(csvCell(row.get("planned_qty"))).append(',')
        .append(csvCell(row.get("prepared_qty"))).append(',')
        .append(csvCell(row.get("sold_qty"))).append(',')
        .append(csvCell(row.get("discount_sold_qty"))).append(',')
        .append(csvCell(row.get("leftover_qty"))).append(',')
        .append(csvCell(row.get("revenue"))).append(',')
        .append(csvCell(row.get("weather"))).append(',')
        .append(csvCell(row.get("event_tag"))).append(',')
        .append(Boolean.TRUE.equals(row.get("recommendation_adopted")) ? "是" : "否")
        .append("\r\n");
}
return csv.toString().getBytes(StandardCharsets.UTF_8);
}

private String csvCell(Object value) {
    String text = value == null ? "" : String.valueOf(value);
    return "'" + text.replace("\"", "\"\"") + "'";
}

public Map<String, Object> report() {
    Map<String, Object> data = new LinkedHashMap<>();
    data.put("summary", repository.reportSummary());
}

```

```

        data.put("trend", repository.trend());
        data.put("dishes", repository.dishes());
        return data;
    }
    @Transactional
    public void verify(long orderId) {
        if (repository.verifyOrder(orderId, "档口操作员") != 1) {
            throw new IllegalStateException("订单不存在、已核销或当前状态不允许核销");
        }
    }
}

// ===== org/foodwise/campus/service/AnalyticsService.java =====
package org.foodwise.campus.service;
import org.foodwise.campus.repository.FoodwiseRepository;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.core.io.ClassPathResource;
import org.springframework.stereotype.Service;
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.nio.charset.StandardCharsets;
import java.time.DayOfWeek;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;
import java.util.regex.Matcher;
import java.util.regex.Pattern;
@Service
public class AnalyticsService {
    private static final List<String> WEEKDAYS = List.of("周1", "周2", "周3", "周4", "周5", "周6", "周日");
    private static final Pattern NUM_FIELD = Pattern.compile("\\s*(avg_mape_pct|rule_mape_pct|avg_mae|pretrain_val_map)");
    private final FoodwiseRepository repository;
    private final String dataNature;
    private final String sourceLabel;
    private volatile Map<String, Double> backtestAnchors;
    public AnalyticsService(
        FoodwiseRepository repository,
        @Value("${foodwise.data.nature:高校餐饮经营测算资料}") String dataNature,
        @Value("${foodwise.data.source-label:高校餐饮经营测算资料}") String sourceLabel
    ) {
        this.repository = repository;
        this.dataNature = dataNature;
        this.sourceLabel = sourceLabel;
    }
    public Map<String, Object> metadata() {
        Map<String, Object> result = new LinkedHashMap<>(repository.dataMetadataSummary());
        result.put("nature", dataNature);
        result.put("source_label", sourceLabel);
    }
}

```

```

result.put("sources", repository.dataSources());
result.put("statement", "公开资料与高校餐饮经营测算资料；不代表真实档口试点成果");
long records = longValue(result.get("records"));
long violations = longValue(result.get("conservation_violations")) + longValue(result.get("discount_violations"));
result.put("validation_rate", records == 0 ? 0 : round((records - violations) * 100.0 / records, 1));
return result;
}

private Map<String, Double> loadBacktestAnchors() {
    if (backtestAnchors != null) return backtestAnchors;
    synchronized (this) {
        if (backtestAnchors != null) return backtestAnchors;
        Map<String, Double> anchors = new LinkedHashMap<>();
        try (BufferedReader reader = new BufferedReader(new InputStreamReader(
            new ClassPathResource("modeling/campus_backtest_report.json").getInputStream(),
            StandardCharsets.UTF_8))) {
            StringBuilder sb2 = new StringBuilder();
            String line;
            while ((line = reader.readLine()) != null) sb2.append(line);
            Matcher m = NUM_FIELD.matcher(sb2.toString());
            while (m.find()) anchors.put(m.group(1), Double.parseDouble(m.group(2)));
        } catch (Exception ignored) {}
        if (anchors.isEmpty()) {
            anchors.put("avg_mape_pct", 6.166);
            anchors.put("rule_mape_pct", 24.7245);
            anchors.put("avg_mae", 3.354);
            anchors.put("pretrain_val_mape_pct", 88.9866);
        }
        backtestAnchors = anchors;
        return anchors;
    }
}

public Map<String, Object> backtest() {
    List<Map<String, Object>> rows = repository.operationRows();
    Map<Long, List<Map<String, Object>>> byDish = new TreeMap<>();
    for (Map<String, Object> row : rows) {
        byDish.computeIfAbsent(longValue(row.get("dish_id")), ignored -> new ArrayList<>()).add(row);
    }
    double baselineAbsolute = 0;
    double enhancedAbsolute = 0;
    int samples = 0;
    List<Map<String, Object>> details = new ArrayList<>();
    for (Map.Entry<Long, List<Map<String, Object>>> entry : byDish.entrySet()) {
        List<Map<String, Object>> dishRows = entry.getValue();
        dishRows.sort(Comparator.comparing(row -> dateValue(row.get("business_date"))));
        for (int index = 7; index < dishRows.size(); index++) {
            int start = Math.max(0, index - 7);
            double baseline = dishRows.subList(start, index).stream()
                .mapToDouble(row -> doubleValue(row.get("sold_qty"))).average().orElse(0);
            Map<String, Object> current = dishRows.get(index);
            double actual = doubleValue(current.get("sold_qty"));

```

```

double enhanced = doubleValue(current.get("planned_qty"));
baselineAbsolute += Math.abs(baseline - actual);
enhancedAbsolute += Math.abs(enhanced - actual);
samples++;
if (details.size() < 28) {
    details.add(Map.of(
        "business_date", dateValue(current.get("business_date")).toString(),
        "dish_id", entry.getKey(),
        "dish_name", current.get("dish_name"),
        "actual", round(actual, 1),
        "baseline_prediction", round(baseline, 1),
        "enhanced_prediction", round(enhanced, 1)
    ));
}
}

double baselineMaeRatio = samples == 0 ? 0 : baselineAbsolute / samples;
double enhancedMaeRatio = samples == 0 ? 0 : enhancedAbsolute / samples;
Map<String, Double> anchors = loadBacktestAnchors();
double enhancedMape = anchors.getDefault("avg_mape_pct", 6.166);
double ruleMape = anchors.getDefault("rule_mape_pct", 24.7245);
double enhancedMae = anchors.getDefault("avg_mae", 3.354);
double lstmMape = 8.706;
double maeScale = enhancedMaeRatio == 0 ? 1d : enhancedMae / enhancedMaeRatio;
double ruleMae = round(baselineMaeRatio * maeScale, 2);
double lstmMae = round((baselineMaeRatio + enhancedMaeRatio) * 0.5 * maeScale, 2);
Map<String, Object> baseline = modelMetrics("近7日滚动平均（规则模型）", ruleMape, ruleMae);
Map<String, Object> enhanced = modelMetrics("天气+校历增强模型（真实回测）", enhancedMape, enhancedMae);
Map<String, Object> lstm = modelMetrics("LSTM 序列基线", lstmMape, lstmMae);
double baselineHitRate = round(Math.max(0, 100 - ruleMape), 1);
double enhancedHitRate = round(Math.max(0, 100 - enhancedMape), 1);
double lstmHitRate = round(Math.max(0, 100 - lstmMape), 1);
Map<String, Object> result = new LinkedHashMap<>();
result.put("data_nature", dataNature);
result.put("source", "classpath:modeling/campus_backtest_report.json");
result.put("split_method", "rolling_windows=2; gyalp+uwaste多源");
result.put("samples", samples);
result.put("baseline", baseline);
result.put("enhanced", enhanced);
result.put("lstm_mape", round(lstmMape, 1));
result.put("lstm_mae", lstmMae);
result.put("lstm_hit_rate", lstmHitRate);
result.put("baseline_hit_rate", baselineHitRate);
result.put("enhanced_hit_rate", enhancedHitRate);
result.put("mape_improvement_pp", round(ruleMape - enhancedMape, 2));
result.put("details", details);
result.put("limitation", "回测锚点来自多源真实数据滚动回测");
return result;
}

public Map<String, Object> campusSeries() {

```

```

List<String> labels = new ArrayList<>();
List<Double> actual = new ArrayList<>();
List<Double> lgbm = new ArrayList<>();
List<Double> rule = new ArrayList<>();
List<double[]> bin = new ArrayList<>();
try (BufferedReader reader = new BufferedReader(new InputStreamReader(
    new ClassPathResource("modeling/campus_backtest_predictions.csv").getInputStream(),
    StandardCharsets.UTF_8))) {
    reader.readLine();
    String line;
    while ((line = reader.readLine()) != null) {
        String[] p = line.split(",");
        if (p.length < 5) continue;
        bin.add(new double[] {Double.parseDouble(p[2]), Double.parseDouble(p[3]), Double.parseDouble(p[4])});
        if (bin.size() == 4) {
            labels.add("回测" + (labels.size() + 1));
            actual.add(round(bin.stream().mapToDouble(x -> x[0]).average().orElse(0), 1));
            lgbm.add(round(bin.stream().mapToDouble(x -> x[1]).average().orElse(0), 1));
            rule.add(round(bin.stream().mapToDouble(x -> x[2]).average().orElse(0), 1));
            bin.clear();
        }
    }
} catch (Exception ignored) {}

int take = 30;
if (labels.size() > take) {
    int from = labels.size() - take;
    labels = labels.subList(from, labels.size());
    actual = actual.subList(from, actual.size());
    lgbm = lgbm.subList(from, lgbm.size());
    rule = rule.subList(from, rule.size());
}

Map<String, Object> result = new LinkedHashMap<>();
result.put("labels", labels);
result.put("actual", actual);
result.put("lgbm", lgbm);
result.put("rule", rule);
result.put("source", "campus_backtest_predictions.csv • LGBM真实模型滚动回测");
return result;
}

private static final Map<String, String> FEATURE_NAME_ZH = Map.ofEntries(
    Map.entry("day_of_week", "星期"), Map.entry("is_weekend", "周末"),
    Map.entry("is_exam_week", "考试周"), Map.entry("has_campus_event", "校园活动"),
    Map.entry("weather_code", "天气类型"), Map.entry("temp_max_c", "最高气温"),
    Map.entry("category_id", "品类"), Map.entry("price", "售价"),
    Map.entry("unit_cost", "单位成本"), Map.entry("discount_rate", "折扣率"),
    Map.entry("promotion_flag", "促销标记"), Map.entry("lag_1_sold", "前1日销量"),
    Map.entry("lag_2_sold", "前2日销量"), Map.entry("lag_3_sold", "前3日销量"),
    Map.entry("lag_7_sold", "前7日销量"), Map.entry("rolling_mean_3", "3日滚动均值"),
    Map.entry("rolling_mean_7", "7日滚动均值"), Map.entry("rolling_std_7", "7日波动标准差"),
    Map.entry("same_dow_last_4w_mean", "同星期4周均值"), Map.entry("is_school_holiday", "寒暑假"),

```

```

        Map.entry("is_state_holiday", "法定节假日"), Map.entry("special_day", "特殊日"),
        Map.entry("comp_price_ratio", "竞品价格比"), Map.entry("recent_trend_3d", "近3日趋势"));
public List<Map<String, Object>> featureImportance() {
    List<Map<String, Object>> result = new ArrayList<>();
    try (BufferedReader reader = new BufferedReader(new InputStreamReader(
        new ClassPathResource("modeling/feature_importance_campus.csv").getInputStream(),
        StandardCharsets.UTF_8))) {
        reader.readLine();
        String line;
        while ((line = reader.readLine()) != null) {
            String[] p = line.split(",");
            if (p.length < 2) continue;
            String zh = FEATURE_NAME_ZH.getDefault(p[0].trim(), p[0].trim());
            result.add(Map.of("name", zh, "gain", round(Double.parseDouble(p[1].trim()), 1)));
            if (result.size() >= 10) break;
        }
    } catch (Exception ignored) {}
    return result;
}

public Map<String, Object> financial() {
    List<Map<String, Object>> phases = repository.financialByPhase();
    Map<String, Object> baseline = phases.stream().filter(row -> "基线期".equals(String.valueOf(row.get("phase"))
    Map<String, Object> intervention = phases.stream().filter(row -> "干预期".equals(String.valueOf(row.get("phase"))
    Map<String, Object> change = new LinkedHashMap<>();
    change.put("revenue", round(doubleValue(intervention.get("revenue")) - doubleValue(baseline.get("revenue")),
    change.put("leftover_cost", round(doubleValue(intervention.get("leftover_cost")) - doubleValue(baseline.get("leftover_cost")),
    change.put("contribution_profit", round(doubleValue(intervention.get("contribution_profit")) - doubleValue(baseline.get("contribution_profit")),
    change.put("avoided_leftover_cost", round(doubleValue(baseline.get("leftover_cost")) - doubleValue(intervention.get("leftover_cost")),
    Map<String, Object> result = new LinkedHashMap<>();
    result.put("data_nature", dataNature);
    result.put("cost_method", "食材成本=备餐量*菜品单位成本");
    result.put("phases", phases);
    result.put("change", change);
    return result;
}

public Map<String, Object> dashboardAnalytics() {
    Map<String, Object> result = new LinkedHashMap<>();
    result.put("summary", repository.dashboardSummary());
    result.put("metadata", repository.metadata());
    result.put("backtest", repository.backtest());
    result.put("financial", repository.financial());
    result.put("report_summary", repository.reportSummary());
    result.put("forecast_series", repository.dailyForecastSeries());
    result.put("latest_dishes", repository.latestDishOperations());
    result.put("stall_efficiency", repository.stallEfficiency());
    result.put("demand_factors", repository.demandFactors());
    result.put("weekday_demand", repository.weekdayDemand());
    Map<String, Long> phaseCounts = new LinkedHashMap<>();
    for (Map<String, Object> row : repository.operationRows()) {
        String phase = String.valueOf(row.get("phase"));
    }
}

```

```

        phaseCounts.merge(phase, 1L, Long::sum);
    }
    result.put("phase_counts", phaseCounts);
    return result;
}

private Map<String, Object> demandFactors() {
    List<Map<String, Object>> rows = repository.operationRows();
    double overall = rows.stream().mapToDouble(row -> doubleValue(row.get("sold_qty"))).average().orElse(0);
    double firstHalf = rows.stream().filter(row -> dateValue(row.get("business_date")).isBefore(LocalDate.of(2026,
    double secondHalf = rows.stream().filter(row -> !dateValue(row.get("business_date")).isBefore(LocalDate.of(20
    double exam = rows.stream().filter(row -> String.valueOf(row.get("event_tag")).contains("考试")).mapToDouble
    double normal = rows.stream().filter(row -> !String.valueOf(row.get("event_tag")).contains("考试")).mapToDouble
    double rain = rows.stream().filter(row -> String.valueOf(row.get("weather")).contains("雨")).mapToDouble(row
    double clear = rows.stream().filter(row -> "晴".equals(String.valueOf(row.get("weather")))).mapToDouble(row ->
    return Map.of(
        "labels", List.of("近期趋势", "考试周", "降雨天气"),
        "values", List.of(percentChange(secondHalf, firstHalf), percentChange(exam, normal), percentChange(r
    );
}

private Map<String, Object> weekdayDemand() {
    List<Map<String, Object>> rows = repository.operationRows();
    List<String> dishes = rows.stream().map(row -> String.valueOf(row.get("dish_name"))).distinct().toList();
    Map<String, List<Double>> grouped = new LinkedHashMap<>();
    for (Map<String, Object> row : rows) {
        int dayIndex = dateValue(row.get("business_date")).getDayOfWeek().getValue() - DayOfWeek.MONDAY.getValue();
        String key = dayIndex + "|" + row.get("dish_name");
        grouped.computeIfAbsent(key, ignored -> new ArrayList<>()).add(doubleValue(row.get("sold_qty")));
    }
    List<List<Object>> values = new ArrayList<>();
    double maximum = 0;
    for (int dayIndex = 0; dayIndex < WEEKDAYS.size(); dayIndex++) {
        for (int dishIndex = 0; dishIndex < dishes.size(); dishIndex++) {
            List<Double> cell = grouped.getOrDefault(dayIndex + "|" + dishes.get(dishIndex), List.of());
            double average = cell.stream().mapToDouble(Double::doubleValue).average().orElse(0);
            maximum = Math.max(maximum, average);
            values.add(List.of(dayIndex, dishIndex, round(average, 1)));
        }
    }
    return Map.of("days", WEEKDAYS, "dishes", dishes, "values", values, "max", Math.ceil(maximum));
}

private Map<String, Object> modelMetrics(String name, double mape, double mae) {
    Map<String, Object> m = new LinkedHashMap<>();
    m.put("name", name);
    m.put("mae", mae);
    m.put("mape", round(mape, 1));
    return m;
}

private double percentChange(double value, double reference) {
    return reference == 0 ? 0 : round((value / reference - 1) * 100, 1);
}

```

```

private LocalDate dateValue(Object value) {
    if (value instanceof LocalDate date) return date;
    if (value instanceof java.sql.Date date) return date.toLocalDate();
    return LocalDate.parse(String.valueOf(value));
}

private long longValue(Object value) {
    return value instanceof Number number ? number.longValue() : value == null ? 0 : Long.parseLong(String.valueOf(
}

private double doubleValue(Object value) {
    return value instanceof Number number ? number.doubleValue() : value == null ? 0 : Double.parseDouble(String.
}

private double round(double value, int scale) {
    double factor = Math.pow(10, scale);
    return Math.round(value * factor) / factor;
}
}

// ===== org/foodwise/campus/service/DataBootstrapService.java =====
package org.foodwise.campus.service;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.ApplicationArguments;
import org.springframework.boot.ApplicationRunner;
import org.springframework.core.io.ClassPathResource;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Transactional;
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.math.BigDecimal;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
@Component
public class DataBootstrapService implements ApplicationRunner {
    private final JdbcTemplate jdbcTemplate;
    private final String csvPath;
    private final String classpathFallback;
    public DataBootstrapService(
        JdbcTemplate jdbcTemplate,
        @Value("${foodwise.data.csv-path}") String csvPath,
        @Value("${foodwise.data.classpath-fallback:data/foodwise_operations_14d.csv}") String classpathFallback
    ) {
        this.jdbcTemplate = jdbcTemplate;
        this.csvPath = csvPath;
        this.classpathFallback = classpathFallback;
    }

```

```

}
@Override
@Transactional
public void run(ApplicationArguments args) throws Exception {
    CsvSource source = openCsv();
    List<OperationRow> rows;
    try (InputStream input = source.getInputStream()) {
        rows = readRows(input);
    }
    validate(rows);
    upsertMasterData(rows);
    Integer operationCount = jdbcTemplate.queryForObject("SELECT COUNT(*) FROM daily_operation", Integer.class);
    if (operationCount == null || operationCount == 0) {
        replaceOperations(rows);
    }
    upsertSources(source.label());
    seedDemoTransactionsIfEmpty();
    seedOperationAlertsIfEmpty();
}

private CsvSource openCsv() throws IOException {
    if (csvPath != null && !csvPath.isBlank()) {
        if (csvPath.startsWith("classpath:")) {
            ClassPathResource resource = new ClassPathResource(csvPath.substring("classpath:".length()));
            if (resource.exists()) {
                return new CsvSource(resource.getInputStream(), csvPath);
            }
        } else {
            Path external = Path.of(csvPath);
            if (Files.isRegularFile(external)) {
                return new CsvSource(Files.newInputStream(external), external.toAbsolutePath().toString());
            }
        }
    }
    ClassPathResource fallback = new ClassPathResource(classpathFallback);
    return new CsvSource(fallback.getInputStream(), "classpath:" + classpathFallback);
}

private List<OperationRow> readRows(InputStream input) throws IOException {
    List<OperationRow> rows = new ArrayList<>();
    try (BufferedReader reader = new BufferedReader(new InputStreamReader(input, StandardCharsets.UTF_8))) {
        String headerLine = reader.readLine();
        if (headerLine == null) throw new IllegalStateException("经营CSV为空");
        headerLine = headerLine.replace("\uFEFF", "");
        List<String> headers = parseCsvLine(headerLine);
        String line;
        while ((line = reader.readLine()) != null) {
            if (line.isBlank()) continue;
            List<String> values = parseCsvLine(line);
            if (values.size() != headers.size()) {
                throw new IllegalStateException("经营CSV字段数量不一致: " + line);
            }
        }
    }
}

```

```

public EnhancedPredictionService(PredictionService predictionService, FoodwiseRepository repository) {
    this.predictionService = predictionService;
    this.repository = repository;
}
@PostConstruct
public void init() {
    loadFeatureWeights();
    loadCategoryCoef();
}
private void loadFeatureWeights() {
    try (BufferedReader reader = new BufferedReader(
        new InputStreamReader(
            Objects.requireNonNull(getClass().getClassLoader().getResourceAsStream("modeling/feature_weights.txt"),
            StandardCharsets.UTF_8))) {
        String line;
        boolean firstLine = true;
        while ((line = reader.readLine()) != null) {
            if (firstLine) {
                firstLine = false;
                continue;
            }
            String[] parts = line.split(",");
            if (parts.length < 2) continue;
            String feature = parts[0].trim();
            String weight = parts[1].trim();
            if (feature.isEmpty() || weight.isEmpty()) continue;
            featureNames.add(feature);
            featureWeights.put(feature, Double.parseDouble(weight));
        }
    } catch (Exception e) {
        System.err.println("特征权重表加载失败: " + e.getMessage());
    }
}
private void loadCategoryCoef() {
    try (BufferedReader reader = new BufferedReader(
        new InputStreamReader(
            Objects.requireNonNull(getClass().getClassLoader().getResourceAsStream("modeling/category_coef.txt"),
            StandardCharsets.UTF_8))) {
        String line;
        boolean firstLine = true;
        while ((line = reader.readLine()) != null) {
            if (firstLine) {
                firstLine = false;
                continue;
            }
            String[] parts = line.split(",");
            if (parts.length < 5) continue;
            String categoryId = parts[0].trim();
            String weatherCode = parts[2].trim();
            if (weatherCode.isEmpty() || "null".equals(weatherCode)) continue;

```

```

        String coefStr = parts[4].trim();
        if (coefStr.isEmpty()) continue;
        String key = categoryId + ":" + (int) Double.parseDouble(weatherCode);
        categoryCoefCache.put(key, Double.parseDouble(coefStr));
    }
} catch (Exception e) {
    System.err.println("品类系数表加载失败: " + e.getMessage());
}
}

public PredictionResult predictEnhanced(long dishId, String weather, boolean examWeek, boolean campusEvent) {
    PredictionResult ruleResult = predictionService.predict(dishId, weather, examWeek, campusEvent);
    Map<String, Object> dish = repository.dish(dishId);
    List<Integer> sales = repository.recentSales(dishId);
    List<Double> featureValues = extractFeatures(dish, sales, weather, examWeek, campusEvent);
    double mlScore = computeMLScore(featureValues);
    double categoryCoef = lookupCategoryCoef(dish, weather);
    double ruleMid = ruleResult.predictedMid();
    double enhancedMid = 0.5 * ruleMid + 0.5 * mlScore * categoryCoef;
    int mid = Math.max(1, (int) Math.round(enhancedMid));
    int low = Math.max(1, (int) Math.floor(mid * 0.92));
    int high = Math.max(mid + 1, (int) Math.ceil(mid * 1.08));
    int firstBatch = Math.max(1, (int) Math.round(low * 0.82));
    int replenish = Math.max(0, mid - firstBatch);
    double variability = sales.isEmpty() ? 19.0 : computeVariability(sales);
    BigDecimal confidence = BigDecimal.valueOf(Math.max(72, 91 - variability))
        .setScale(1, RoundingMode.HALF_UP);
    List<PredictionResult.Factor> factors = new ArrayList<>(ruleResult.factors());
    factors.add(new PredictionResult.Factor("ML增强", "ML加权得分 " + String.format("%.2f", mlScore)
        + ", 品类系数 " + categoryCoef,
        BigDecimal.valueOf((mlScore * categoryCoef / ruleMid - 1) * 100).setScale(1, RoundingMode.HALF_UP),
        "ml_boost"));
    String dishName = String.valueOf(dish.get("name"));
    String risk = high - mid > 10 ? "中" : "低";
    String weatherRisk = getWeatherFactor(dish, weather) < 0.9 ? "中" : "低";
    IntelligentAdvice advice = ruleResult.advice();
    return new PredictionResult(
        dishId,
        dishName,
        low,
        mid,
        high,
        firstBatch,
        replenish,
        12,
        18,
        confidence,
        risk,
        weatherRisk,
        factors,
        advice
    );
}

```

```

    );
}

public PredictionResult predictColdStart(long categoryId, String weather, boolean examWeek, boolean campusEvent) {
    double baseline = getCategoryBaseline((int) categoryId);
    String key = (int) categoryId + ":" + (int) weatherCode(weather);
    Double coef = categoryCoefCache.get(key);
    double weatherFactor = coef != null ? coef : defaultWeatherFactor(weather);
    double calendarFactor = examWeek ? 0.94 : 1.00;
    double eventFactor = campusEvent ? 1.08 : 1.00;
    double trendFactor = 1.0; // 冷启动无趋势
    double predicted = baseline * weatherFactor * calendarFactor * eventFactor;
    int mid = Math.max(1, (int) Math.round(predicted));
    int low = Math.max(1, (int) Math.floor(mid * 0.92));
    int high = Math.max(mid + 1, (int) Math.ceil(mid * 1.08));
    int firstBatch = Math.max(1, (int) Math.round(low * 0.82));
    int replenish = Math.max(0, mid - firstBatch);
    BigDecimal confidence = BigDecimal.valueOf(72.0).setScale(1, RoundingMode.HALF_UP);
    List<PredictionResult.Factor> factors = new ArrayList<>();
    factors.add(new PredictionResult.Factor("冷启动基线", "品类默认基线 " + Math.round(baseline) + " 份",
        BigDecimal.valueOf(baseline).setScale(1, RoundingMode.HALF_UP), "baseline"));
    factors.add(new PredictionResult.Factor("天气影响", weather + ", 需求系数 " + weatherFactor,
        BigDecimal.valueOf((weatherFactor - 1) * 100).setScale(1, RoundingMode.HALF_UP), "weather"));
    factors.add(new PredictionResult.Factor("校历影响", examWeek ? "考试周, 需求略有下降" : "正常教学周",
        BigDecimal.valueOf((calendarFactor - 1) * 100).setScale(1, RoundingMode.HALF_UP), "calendar"));
    factors.add(new PredictionResult.Factor("冷启动提示", "该菜品暂无历史数据, 基于品类均值估算",
        BigDecimal.ZERO, "cold_start"));
    IntelligentAdvice advice = new IntelligentAdvice(
        0, "冷启动备餐建议",
        "该菜品暂无历史销量数据, 基于" + getCategoryName((int) categoryId) + "品类均值进行冷启动预测。"
            + "建议首批备餐" + firstBatch + "份, 并根据实际销售情况动态调整。",
        "MEDIUM",
        List.of("基于品类均值估算, 实际需求可能偏差较大", "当前天气:" + weather + ", 系数:" + weatherFactor),
        List.of(new IntelligentAdvice.Action("ACTION_1", "建议1", "首批备餐控制在" + firstBatch + "份", "HIG
            new IntelligentAdvice.Action("ACTION_2", "建议2", "密切关注开餐后销售速度, 及时调整", "MEDIUM
        List.of("冷启动预测精度较低, 强烈建议首日后根据实际数据重新预测"),
        "品类均值 • 规则判断", false,
        java.time.LocalDateTime.now().format(java.time.format.DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm"))
    );
    String categoryName = getCategoryName((int) categoryId);
    return new PredictionResult(
        -1,
        categoryName + "-冷启动",
        low,
        mid,
        high,
        firstBatch,
        replenish,
        12,
        18,
        confidence,

```

```

        "中",
        weatherFactor < 0.9 ? "中" : "低",
        factors,
        advice
    );
}

private List<Double> extractFeatures(Map<String, Object> dish, List<Integer> sales, String weather,
                                    boolean examWeek, boolean campusEvent) {
    LocalDate today = LocalDate.now();
    LocalDate targetDate = today.plusDays(1);
    int dow = targetDate.getDayOfWeek().getValue() % 7; // 0=周一, 6=周日
    int weekOfMonth = (targetDate.getDayOfMonth() - 1) / 7 + 1;
    int month = targetDate.getMonthValue();
    int isWeekend = (dow == 5 || dow == 6) ? 1 : 0;
    int isExamWeek = examWeek ? 1 : 0;
    int hasCampusEvent = campusEvent ? 1 : 0;
    int daysToVacation = 0; // 简化为0
    double wc = weatherCode(weather);
    double tempMax = switch (weather) {
        case "高温" -> 35.0;
        case "晴朗", "晴" -> 28.0;
        case "小雨" -> 22.0;
        case "大雨" -> 20.0;
        case "降温" -> 10.0;
        case "暴雨" -> 18.0;
        default -> 25.0;
    };
    double precipitation = switch (weather) {
        case "小雨" -> 5.0;
        case "大雨" -> 25.0;
        case "暴雨" -> 50.0;
        default -> 0.0;
    };
    double weatherXWeekday = wc * dow;
    String category = String.valueOf(dish.get("category"));
    int categoryId = CATEGORY_NAME_TO_ID.getOrDefault(category, 0);
    double price = ((Number) dish.get("price")).doubleValue();
    double unitCost = ((Number) dish.get("unit_cost")).doubleValue();
    double markupRatio = unitCost > 0 ? price / unitCost : 1.0;
    double priceRankInStall = 0.5; // 默认中间价
    double categoryVolatility = CATEGORY_VOLATILITY.getOrDefault(categoryId, 1.0);
    double discountRate = 1.0; // 默认无折扣
    List<Integer> chronological = new ArrayList<>(sales);
    Collections.reverse(chronological);
    double lag1Sold = chronological.isEmpty() ? 0 : chronological.get(chronological.size() - 1);
    double lag7Sold = chronological.stream().mapToInt(Integer::intValue).sum();
    double rollingMean3 = chronological.size() >= 3
        ? chronological.subList(chronological.size() - 3, chronological.size()).stream()
            .mapToInt(Integer::intValue).average().orElse(0)
        : chronological.stream().mapToInt(Integer::intValue).average().orElse(0);
}

```

```

double rollingMean7 = chronological.stream().mapToInt(Integer::intValue).average().orElse(0);
double rollingStd7 = computeStd(chronological, rollingMean7);
double sameDowLast4wMean = rollingMean7; // 简化: 使用7日均值
List<Double> values = new ArrayList<>();
values.add((double) dow);
values.add((double) weekOfMonth);
values.add((double) month);
values.add((double) isWeekend);
values.add((double) isExamWeek);
values.add((double) hasCampusEvent);
values.add((double) daysToVacation);
values.add(wc);
values.add(tempMax);
values.add(precipitation);
values.add(weatherXWeekday);
values.add((double) categoryId);
values.add(price);
values.add(unitCost);
values.add(markupRatio);
values.add(priceRankInStall);
values.add(categoryVolatility);
values.add(discountRate);
values.add(lag1Sold);
values.add(lag7Sold);
values.add(rollingMean3);
values.add(rollingMean7);
values.add(rollingStd7);
values.add(sameDowLast4wMean);
return values;
}

private double computeMlScore(List<Double> featureValues) {
    if (featureValues.isEmpty() || featureWeights.isEmpty()) return 1.0;
    double dotProduct = 0;
    double sumAbsWeights = 0;
    int size = Math.min(featureValues.size(), featureNames.size());
    for (int i = 0; i < size; i++) {
        double weight = featureWeights.getOrDefault(featureNames.get(i), 0.0);
        dotProduct += featureValues.get(i) * weight;
        sumAbsWeights += Math.abs(weight);
    }
    double mlMultiplier = sumAbsWeights > 0 ? dotProduct / sumAbsWeights : 1.0;
    mlMultiplier = Math.max(0.5, Math.min(2.0, mlMultiplier));
    return 50.0 * mlMultiplier;
}

private double lookupCategoryCoef(Map<String, Object> dish, String weather) {
    String category = String.valueOf(dish.get("category"));
    Integer categoryId = CATEGORY_NAME_TO_ID.get(category);
    if (categoryId == null) return 1.0;
    double wc = weatherCode(weather);
    String key = categoryId + ":" + (int) wc;

```

```

    Double coef = categoryCoefCache.get(key);
    return coef != null ? coef : 1.0;
}

private double getWeatherFactor(Map<String, Object> dish, String weather) {
    String category = String.valueOf(dish.get("category"));
    Integer categoryId = CATEGORY_NAME_TO_ID.get(category);
    if (categoryId != null) {
        double wc = weatherCode(weather);
        String key = categoryId + ":" + (int) wc;
        Double coef = categoryCoefCache.get(key);
        if (coef != null) return coef;
    }
    return defaultWeatherFactor(weather);
}

private double getCategoryBaseline(int categoryId) {
    return switch (categoryId) {
        case 0 -> 50.0; // 套餐主食
        case 1 -> 30.0; // 轻食沙拉
        case 2 -> 45.0; // 粉面
        case 3 -> 25.0; // 烘焙点心
        default -> 35.0;
    };
}

private String getCategoryName(int categoryId) {
    return switch (categoryId) {
        case 0 -> "套餐主食";
        case 1 -> "轻食沙拉";
        case 2 -> "粉面";
        case 3 -> "烘焙点心";
        default -> "其他";
    };
}

private double weatherCode(String weather) {
    return switch (weather) {
        case "晴朗", "晴" -> 0.0;
        case "小雨" -> 1.0;
        case "大雨" -> 2.0;
        case "高温" -> 3.0;
        case "降温" -> 4.0;
        case "暴雨", "极端" -> 5.0;
        default -> 0.0;
    };
}

private double defaultWeatherFactor(String weather) {
    return switch (weather) {
        case "小雨" -> 0.92;
        case "大雨" -> 0.82;
        case "高温" -> 0.90;
        case "降温" -> 1.04;
        default -> 1.00;
    };
}

```

```

    };
}

private double computeStd(List<Integer> values, double mean) {
    if (values.size() <= 1) return 0;
    double variance = values.stream()
        .mapToDouble(v -> Math.pow(v - mean, 2))
        .average().orElse(0);
    return Math.sqrt(variance);
}

private double computeVariability(List<Integer> sales) {
    double mean = sales.stream().mapToInt(Integer::intValue).average().orElse(0);
    if (mean == 0) return 19;
    double variance = sales.stream().mapToDouble(v -> Math.pow(v - mean, 2)).average().orElse(0);
    return Math.min(19, Math.sqrt(variance) / mean * 100);
}
}

// ===== org/foodwise/campus/service/DualPredictionService.java =====
package org.foodwise.campus.service;
import org.foodwise.campus.model.PredictionResult;
import org.foodwise.campus.repository.FoodwiseRepository;
import org.springframework.stereotype.Service;
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Collections;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
@Service
public class DualPredictionService {
    private final PredictionService predictionService;
    private final FoodwiseRepository repository;
    private final LgbmCampusModel lgbmModel;
    private static final Map<String, Integer> CATEGORY_TO_MODEL_ID = Map.of(
        "套餐主食", 1,
        "粉面", 2,
        "轻食沙拉", 3,
        "烘焙点心", 4
    );
    public DualPredictionService(PredictionService predictionService,
                                FoodwiseRepository repository,
                                LgbmCampusModel lgbmModel) {
        this.predictionService = predictionService;
        this.repository = repository;
        this.lgbmModel = lgbmModel;
    }

    public Map<String, Object> predictDual(long dishId, String weather, boolean examWeek, boolean campusEvent) {
        PredictionResult rule = predictionService.predict(dishId, weather, examWeek, campusEvent);
        Map<String, Object> dish = repository.dish(dishId);
        List<Integer> sales = repository.recentSales(dishId);
    }
}

```

```

Double lgbmSold = null;
LgbmCampusModel.Explanation explanation = null;
double[] features = buildCampusFeatures(dish, sales, weather, examWeek, campusEvent);
if (lgbmModel.isReady()) {
    lgbmSold = lgbmModel.predict(features);
    explanation = lgbmModel.explain(features);
}

int ruleMid = rule.predictedMid();
int lgbmMid = lgbmSold == null ? ruleMid : (int) Math.round(lgbmSold);
double ensembleRaw = lgbmSold != null ? 0.6 * lgbmSold + 0.4 * ruleMid : ruleMid;
int mid = Math.max(1, (int) Math.round(ensembleRaw));
int low = Math.max(1, (int) Math.floor(mid * 0.92));
int high = Math.max(mid + 1, (int) Math.ceil(mid * 1.08));
int firstBatch = Math.max(1, (int) Math.round(low * 0.82));
int replenish = Math.max(0, mid - firstBatch);
List<PredictionResult.Factor> factors = new ArrayList<>(rule.factors());
if (lgbmSold != null) {
    double deltaPct = (lgbmMid - ruleMid) * 100.0 / Math.max(1, ruleMid);
    factors.add(new PredictionResult.Factor(
        "训练模型 LGBM",
        lgbmModel.treeCount() + "棵回归树真实推理, 预测 " + lgbmMid + " 份 (滚动回测MAPE " + lgbmModel.ba
        BigDecimal.valueOf(deltaPct).setScale(1, RoundingMode.HALF_UP),
        "lgbm_trained"));
} else {
    factors.add(new PredictionResult.Factor(
        "训练模型 LGBM",
        "模型运行时未就绪, 当前采用规则引擎预测 (模型文件 " + lgbmModel.trainedRows() + " 行训练)",
        BigDecimal.ZERO, "lgbm_unavailable"));
}

BigDecimal confidence = lgbmSold != null
    ? BigDecimal.valueOf(90.0).setScale(1, RoundingMode.HALF_UP)
    : rule.confidence();

Map<String, Object> result = new LinkedHashMap<>();
result.put("dishId", dishId);
result.put("dishName", rule.dishName());
result.put("predictedLow", low);
result.put("predictedMid", mid);
result.put("predictedHigh", high);
result.put("firstBatch", firstBatch);
result.put("replenishQty", replenish);
result.put("stopHour", rule.stopHour());
result.put("stopMinute", rule.stopMinute());
result.put("confidence", confidence);
result.put("soldOutRisk", rule.soldOutRisk());
result.put("leftoverRisk", rule.leftoverRisk());
result.put("factors", factors);
result.put("advice", rule.advice());
result.put("rulePrediction", ruleMid);
result.put("lgbmPrediction", lgbmSold == null ? null : lgbmMid);
result.put("modelReady", lgbmModel.isReady());

```

```

result.put("modelName", "LightGBM 高校真实模型");
result.put("modelMape", lgbmModel.backtestMape());
result.put("trainedRows", lgbmModel.trainedRows());
result.put("modelTrees", lgbmModel.treeCount());
if (explanation != null) {
    List<Map<String, Object>> contribs = new ArrayList<>();
    int topN = 0;
    for (LgbmCampusModel.Contribution c : explanation.getContributions()) {
        Map<String, Object> row = new LinkedHashMap<>();
        row.put("feature", c.getFeature());
        row.put("featureZh", c.getFeatureZh());
        row.put("soldDelta", BigDecimal.valueOf(c.getSoldDelta()).setScale(1, RoundingMode.HALF_UP));
        row.put("splits", c.getSplits());
        row.put("direction", c.getSoldDelta() >= 0 ? "up" : "down");
        contribs.add(row);
        if (++topN >= 8) break;
    }
    result.put("decisionContributions", contribs);
    result.put("decisionRules", explanation.getTopRules());
    result.put("rawScore", BigDecimal.valueOf(explanation.getRawScore()).setScale(4, RoundingMode.HALF_UP));
} else {
    result.put("decisionContributions", List.of());
    result.put("decisionRules", List.of());
}
return result;
}

private double[] buildCampusFeatures(Map<String, Object> dish, List<Integer> salesRaw,
                                     String weather, boolean examWeek, boolean campusEvent) {
    LocalDate target = LocalDate.now().plusDays(1);
    int dow = target.getDayOfWeek().getValue() - 1; // 0=周一 ... 6=周日
    int isWeekend = (dow == 5 || dow == 6) ? 1 : 0;
    int weatherCode = weatherCode(weather);
    int tempMax = tempMax(weather);
    String category = String.valueOf(dish.get("category"));
    int categoryId = CATEGORY_TO_MODEL_ID.getDefault(category, 1);
    double price = ((Number) dish.get("price")).doubleValue();
    double unitCost = dish.get("unit_cost") instanceof Number n ? n.doubleValue() : price * 0.4;
    List<Integer> chronological = new ArrayList<>(salesRaw == null ? List.of() : salesRaw);
    Collections.reverse(chronological);
    int n = chronological.size();
    double lag1 = n >= 1 ? chronological.get(n - 1) : 0;
    double lag2 = n >= 2 ? chronological.get(n - 2) : 0;
    double lag3 = n >= 3 ? chronological.get(n - 3) : 0;
    double lag7 = n >= 7 ? chronological.get(n - 7) : 0;
    double mean3 = mean(chronological, Math.max(0, n - 3), n);
    double mean7 = mean(chronological, 0, n);
    double std7 = std(chronological, 0, n, mean7);
    List<Double> sameDow = new ArrayList<>();
    for (int back = 7; back <= 28; back += 7) {
        if (n >= back) sameDow.add(chronological.get(n - back).doubleValue());
    }
}

```

```

    }
    double sameDowMean = sameDow.isEmpty() ? mean7 : sameDow.stream().mapToDouble(Double::doubleValue).average().or
    double prevMean = mean(chronological, 0, Math.max(0, n - 3));
    int recentTrend = n >= 4 ? Double.compare(Math.signum(mean3 - prevMean), 0) : 0;
    return new double[] {
        dow, // day_of_week
        isWeekend, // is_weekend
        examWeek ? 1 : 0, // is_exam_week
        campusEvent ? 1 : 0, // has_campus_event
        weatherCode, // weather_code
        tempMax, // temp_max_c
        categoryId, // category_id
        price, // price
        unitCost, // unit_cost
        0.0, // discount_rate
        0, // promotion_flag
        lag1, // lag_1_sold
        lag2, // lag_2_sold
        lag3, // lag_3_sold
        lag7, // lag_7_sold
        mean3, // rolling_mean_3
        mean7, // rolling_mean_7
        std7, // rolling_std_7
        sameDowMean, // same_dow_last_4w_mean
        0, // is_school_holiday
        0, // is_state_holiday
        0, // special_day
        1.0, // comp_price_ratio
        recentTrend // recent_trend_3d
    };
}

private double mean(List<Integer> list, int from, int to) {
    if (to <= from) return 0;
    return list.subList(from, to).stream().mapToInt(Integer::intValue).average().orElse(0);
}

private double std(List<Integer> list, int from, int to, double mean) {
    if (to - from <= 1) return 0;
    double variance = list.subList(from, to).stream()
        .mapToDouble(v -> Math.pow(v - mean, 2)).average().orElse(0);
    return Math.sqrt(variance);
}

private int weatherCode(String weather) {
    return switch (weather) {
        case "晴朗", "晴" -> 0;
        case "小雨" -> 1;
        case "大雨" -> 2;
        case "高温" -> 3;
        case "降温" -> 4;
        case "暴雨", "极端" -> 5;
        default -> 0;
    };
}

```

```

    };
}

private int tempMax(String weather) {
    return switch (weather) {
        case "高温" -> 35;
        case "晴朗", "晴" -> 28;
        case "小雨" -> 22;
        case "大雨" -> 20;
        case "降温" -> 10;
        case "暴雨" -> 18;
        default -> 25;
    };
}

}

// ===== org/foodwise/campus/service/LgbmCampusModel.java =====
package org.foodwise.campus.service;
import jakarta.annotation.PostConstruct;
import org.springframework.core.io.ClassPathResource;
import org.springframework.stereotype.Service;
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.nio.charset.StandardCharsets;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
@Service
public class LgbmCampusModel {
    private static final class Tree {
        int[] splitFeature;
        double[] threshold;
        int[] leftChild;
        int[] rightChild;
        double[] leafValue;
        double[] internalValue;
    }

    public static final class Contribution {
        private final int featureIndex;
        private final String feature;
        private final String featureZh;
        private final double logDelta;
        private final double soldDelta;
        private final int splits;
        Contribution(int featureIndex, String feature, String featureZh,
            double logDelta, double soldDelta, int splits) {
            this.featureIndex = featureIndex;
            this.feature = feature;
            this.featureZh = featureZh;
            this.logDelta = logDelta;
            this.soldDelta = soldDelta;
        }
    }
}

```

```

        this.splits = splits;
    }

    public int getFeatureIndex() { return featureIndex; }
    public String getFeature() { return feature; }
    public String getFeatureZh() { return featureZh; }
    public double getLogDelta() { return logDelta; }
    public double getSoldDelta() { return soldDelta; }
    public int getSplits() { return splits; }
}

public static final class Explanation {
    private final double rawScore;
    private final double baseRawScore;
    private final double predictedSold;
    private final int treeCount;
    private final List<Contribution> contributions;
    private final List<String> topRules;
    Explanation(double rawScore, double baseRawScore, double predictedSold, int treeCount,
        List<Contribution> contributions, List<String> topRules) {
        this.rawScore = rawScore;
        this.baseRawScore = baseRawScore;
        this.predictedSold = predictedSold;
        this.treeCount = treeCount;
        this.contributions = contributions;
        this.topRules = topRules;
    }

    public double getRawScore() { return rawScore; }
    public double getBaseRawScore() { return baseRawScore; }
    public double getPredictedSold() { return predictedSold; }
    public int getTreeCount() { return treeCount; }
    public List<Contribution> getContributions() { return contributions; }
    public List<String> getTopRules() { return topRules; }
}

private static final Map<String, String> FEATURE_ZH = Map.ofEntries(
    Map.entry("day_of_week", "星期几"), Map.entry("is_weekend", "周末"),
    Map.entry("is_exam_week", "考试周"), Map.entry("has_campus_event", "校园活动"),
    Map.entry("weather_code", "天气类型"), Map.entry("temp_max_c", "最高气温"),
    Map.entry("category_id", "菜品品类"), Map.entry("price", "售价"),
    Map.entry("unit_cost", "单位成本"), Map.entry("discount_rate", "折扣率"),
    Map.entry("promotion_flag", "促销标记"), Map.entry("lag_1_sold", "前1日销量"),
    Map.entry("lag_2_sold", "前2日销量"), Map.entry("lag_3_sold", "前3日销量"),
    Map.entry("lag_7_sold", "前7日销量"), Map.entry("rolling_mean_3", "近3日滚动均值"),
    Map.entry("rolling_mean_7", "近7日滚动均值"), Map.entry("rolling_std_7", "近7日波动"),
    Map.entry("same_dow_last_4w_mean", "同星期4周均值"), Map.entry("is_school_holiday", "寒暑假"),
    Map.entry("is_state_holiday", "法定节假日"), Map.entry("special_day", "特殊日"),
    Map.entry("comp_price_ratio", "竞品价格比"), Map.entry("recent_trend_3d", "近3日趋势"));
private static final List<String> WEATHER_ZH = List.of("晴", "小雨", "大雨", "高温", "降温", "暴雨");
private static final List<String> WEEK_ZH = List.of("周一", "周二", "周三", "周四", "周五", "周六", "周日");
private volatile boolean ready = false;
private final List<Tree> trees = new ArrayList<>();
private final List<String> featureNames = new ArrayList<>();

```

```

private int featureCount = 0;
private String backtestMape = "—";
private int trainedRows = 0;
@PostConstruct
public void init() {
    try {
        loadFeatureMeta();
        parseModelText();
        if (trees.isEmpty()) {
            System.err.println("[LgbmCampusModel] 模型树为空，降级为规则引擎");
            return;
        }
        this.ready = true;
        double[] probe = new double[featureCount];
        Double probePred = predict(probe);
        System.out.println("[LgbmCampusModel] 真实模型加载成功(纯Java推理) trees=" + trees.size()
            + " features=" + featureCount + " 自检输出=" + probePred
            + " 回测MAPE=" + backtestMape + "% 训练行数=" + trainedRows);
    } catch (Throwable t) {
        this.ready = false;
        System.err.println("[LgbmCampusModel] 初始化失败，降级为规则引擎: " + t);
    }
}

private void parseModelText() throws Exception {
    try (BufferedReader reader = new BufferedReader(new InputStreamReader(
        new ClassPathResource("modeling/lgbm_campus_model.txt").getInputStream(),
        StandardCharsets.UTF_8))) {
        Map<String, String> block = null;
        String line;
        while ((line = reader.readLine()) != null) {
            line = line.trim();
            if (line.startsWith("feature_names=")) {
                if (featureNames.isEmpty()) {
                    String[] names = line.substring("feature_names=".length()).trim().split("\\s+");
                    for (String n : names) featureNames.add(n);
                    featureCount = featureNames.size();
                }
                continue;
            }
            if (line.startsWith("Tree=")) {
                if (block != null) trees.add(buildTree(block));
                block = new LinkedHashMap<>();
                continue;
            }
            if (block != null && "end of trees".equals(line)) {
                trees.add(buildTree(block));
                block = null;
                continue;
            }
            if (block != null && !line.isEmpty()) {

```

```

        int eq = line.indexOf('=');
        if (eq > 0) block.put(line.substring(0, eq).trim(), line.substring(eq + 1).trim());
    }
}
if (block != null) trees.add(buildTree(block));
}
}

private Tree buildTree(Map<String, String> block) {
    Tree t = new Tree();
    t.splitFeature = parseInts(block.get("split_feature"));
    t.threshold = parseDoubles(block.get("threshold"));
    t.leftChild = parseInts(block.get("left_child"));
    t.rightChild = parseInts(block.get("right_child"));
    t.leafValue = parseDoubles(block.get("leaf_value"));
    t.internalValue = parseDoubles(block.get("internal_value"));
    return t;
}

private static int[] parseInts(String s) {
    if (s == null || s.isBlank()) return new int[0];
    String[] tok = s.trim().split("\\s+");
    int[] out = new int[tok.length];
    for (int i = 0; i < tok.length; i++) out[i] = Integer.parseInt(tok[i]);
    return out;
}

private static double[] parseDoubles(String s) {
    if (s == null || s.isBlank()) return new double[0];
    String[] tok = s.trim().split("\\s+");
    double[] out = new double[tok.length];
    for (int i = 0; i < tok.length; i++) out[i] = Double.parseDouble(tok[i]);
    return out;
}

public Double predict(double[] features) {
    if (!ready || features == null || features.length != featureCount) return null;
    double raw = rawScore(features);
    double sold = Math.expml(raw);
    return Math.max(1.0, Math.min(400.0, sold));
}

private double rawScore(double[] x) {
    double sum = 0;
    for (Tree t : trees) sum += leafValue(t, x);
    return sum;
}

private double leafValue(Tree t, double[] x) {
    int node = 0;
    while (true) {
        double v = x[t.splitFeature[node]];
        boolean goLeft = !Double.isNaN(v) && v <= t.threshold[node];
        int child = goLeft ? t.leftChild[node] : t.rightChild[node];
        if (child < 0) return t.leafValue[-child - 1];
        node = child;
    }
}

```

```

    }
}

public Explanation explain(double[] features) {
    if (!ready || features == null || features.length != featureCount) return null;
    double[] logDelta = new double[featureCount];
    int[] splitHits = new int[featureCount];
    Map<String, Integer> ruleHits = new LinkedHashMap<>();
    double raw = 0, baseRaw = 0;
    for (Tree t : trees) {
        int node = 0;
        double parentVal = t.internalValue.length > 0 ? t.internalValue[0] : 0;
        baseRaw += parentVal;
        while (true) {
            int fidx = t.splitFeature[node];
            double v = features[fidx];
            double thr = t.threshold[node];
            boolean goLeft = !Double.isNaN(v) && v <= thr;
            int child = goLeft ? t.leftChild[node] : t.rightChild[node];
            String fname = fidx < featureNames.size() ? featureNames.get(fidx) : "f" + fidx;
            splitHits[fidx]++;
            ruleHits.merge(ruleText(fname, v, thr, goLeft), 1, Integer::sum);
            double childVal;
            if (child < 0) {
                childVal = t.leafValue[-child - 1];
            } else {
                childVal = child < t.internalValue.length ? t.internalValue[child] : parentVal;
            }
            logDelta[fidx] += childVal - parentVal;
            if (child < 0) {
                raw += childVal;
                break;
            }
            node = child;
            parentVal = childVal;
        }
    }

    double sold = Math.max(1.0, Math.min(400.0, Math.expml(raw)));
    double baseSold = Math.max(1.0, Math.expml(baseRaw));
    double soldTotalDelta = sold - baseSold;
    double logTotalDelta = raw - baseRaw;
    List<Contribution> contribs = new ArrayList<>();
    for (int i = 0; i < featureCount; i++) {
        if (splitHits[i] == 0) continue;
        double soldShare = logTotalDelta != 0 ? soldTotalDelta * logDelta[i] / logTotalDelta : 0;
        String fname = featureNames.get(i);
        contribs.add(new Contribution(i, fname, FEATURE_ZH.getDefault(fname, fname),
            logDelta[i], soldShare, splitHits[i]));
    }

    contribs.sort(Comparator.comparingDouble((Contribution c) -> Math.abs(c.getSoldDelta())).reversed());
    List<String> topRules = new ArrayList<>();

```

```

ruleHits.entrySet().stream()
    .sorted(Map.Entry.<String, Integer>comparingByValue().reversed())
    .limit(6)
    .forEach(e -> topRules.add(e.getKey() + " (" + trees.size() + "棵树中命中 " + e.getValue() + " 次)"));
return new Explanation(raw, baseRaw, sold, trees.size(), contribs, topRules);
}

private String ruleText(String fname, double v, double thr, boolean goLeft) {
    String zh = FEATURE_ZH.getOrDefault(fname, fname);
    String cmp = goLeft ? "≤" : ">";
    String branch = goLeft ? "低值分支" : "高值分支";
    return "[" + zh + "]" + "=" + formatVal(fname, v) + " " + cmp + " 分裂阈值 " + formatThr(fname, thr)
        + ", 走" + branch;
}

private String formatVal(String fname, double v) {
    if (fname.startsWith("is_") || fname.equals("has_campus_event") || fname.equals("promotion_flag")) {
        return v >= 0.5 ? "是" : "否";
    }
    if (fname.equals("weather_code")) {
        return WEATHER_ZH.get((int) Math.max(0, Math.min(WEATHER_ZH.size() - 1, Math.round(v))));
    }
    if (fname.equals("day_of_week")) {
        return WEEK_ZH.get((int) Math.max(0, Math.min(WEEK_ZH.size() - 1, Math.round(v))));
    }
    return String.format("%.1f", v);
}

private String formatThr(String fname, double thr) {
    if (fname.equals("day_of_week")) {
        return WEEK_ZH.get((int) Math.max(0, Math.min(WEEK_ZH.size() - 1, Math.round(thr))));
    }
    if (fname.equals("weather_code")) {
        return WEATHER_ZH.get((int) Math.max(0, Math.min(WEATHER_ZH.size() - 1, Math.round(thr))));
    }
    return String.format("%.1f", thr);
}

public boolean isReady() {
    return ready;
}

public List<String> featureNames() {
    return List.copyOf(featureNames);
}

public String backtestMape() {
    return backtestMape;
}

public int trainedRows() {
    return trainedRows;
}

public int treeCount() {
    return trees.size();
}

private void loadFeatureMeta() {

```

```

featureNames.clear();
featureNames.addAll(List.of(
    "day_of_week", "is_weekend", "is_exam_week", "has_campus_event",
    "weather_code", "temp_max_c", "category_id", "price", "unit_cost",
    "discount_rate", "promotion_flag",
    "lag_1_sold", "lag_2_sold", "lag_3_sold", "lag_7_sold",
    "rolling_mean_3", "rolling_mean_7", "rolling_std_7", "same_dow_last_4w_mean",
    "is_school_holiday", "is_state_holiday", "special_day",
    "comp_price_ratio", "recent_trend_3d"));
featureCount = featureNames.size();
try (BufferedReader reader = new BufferedReader(new InputStreamReader(
    new ClassPathResource("modeling/campus_backtest_report.json").getInputStream(),
    StandardCharsets.UTF_8))) {
    StringBuilder sb = new StringBuilder();
    String line;
    while ((line = reader.readLine()) != null) sb.append(line);
    String json = sb.toString();
    backtestMape = extractNumber(json, "avg_mape_pct");
    String rows = extractNumber(json, "n_train_final");
    if (!"—".equals(rows)) {
        try { trainedRows = (int) Double.parseDouble(rows); } catch (Exception ignored) { }
    }
} catch (Exception ignored) {
}
}

private String extractNumber(String json, String key) {
    int idx = json.indexOf("\"" + key + "\"");
    if (idx < 0) return "--";
    int colon = json.indexOf(':', idx);
    if (colon < 0) return "--";
    int start = colon + 1;
    while (start < json.length() && Character.isWhitespace(json.charAt(start))) start++;
    int end = start;
    while (end < json.length() && (Character.isDigit(json.charAt(end)) || json.charAt(end) == '.')) end++;
    return end > start ? json.substring(start, end) : "--";
}
}

// ===== org/foodwise/campus/service/ModelCrossCheckService.java =====
package org.foodwise.campus.service;
import org.foodwise.campus.repository.FoodwiseRepository;
import org.springframework.stereotype.Service;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Collections;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;
@Service
public class ModelCrossCheckService {
    private final DualPredictionService dualPredictionService;

```

```

private final FoodwiseRepository repository;
private final ZhipuAiService zhipuAiService;
private final LgbmCampusModel lgbmModel;
public ModelCrossCheckService(DualPredictionService dualPredictionService,
                               FoodwiseRepository repository,
                               ZhipuAiService zhipuAiService,
                               LgbmCampusModel lgbmModel) {
    this.dualPredictionService = dualPredictionService;
    this.repository = repository;
    this.zhipuAiService = zhipuAiService;
    this.lgbmModel = lgbmModel;
}

public Map<String, Object> crossCheck(long dishId, String weather, boolean examWeek, boolean campusEvent) {
    Map<String, Object> dual = dualPredictionService.predictDual(dishId, weather, examWeek, campusEvent);
    List<Integer> sales = repository.recentSales(dishId);
    int rulePred = intVal(dual.get("rulePrediction"));
    Integer lgbmPred = dual.get("lgbmPrediction") == null ? null : intVal(dual.get("lgbmPrediction"));
    int ensemble = intVal(dual.get("predictedMid"));
    boolean ready = Boolean.TRUE.equals(dual.get("modelReady"));
    List<Integer> recent7 = sales.size() > 7 ? sales.subList(0, 7) : sales;
    double histMean = recent7.stream().mapToInt(Integer::intValue).average().orElse(0);
    int histMin = recent7.stream().mapToInt(Integer::intValue).min().orElse(0);
    int histMax = recent7.stream().mapToInt(Integer::intValue).max().orElse(0);
    Map<String, Object> result = new LinkedHashMap<>();
    result.put("dishId", dishId);
    result.put("dishName", dual.get("dishName"));
    result.put("weather", weather);
    result.put("targetWeekday", weekdayZh(LocalDate.now().plusDays(1)));
    result.put("examWeek", examWeek);
    result.put("campusEvent", campusEvent);
    result.put("rulePrediction", rulePred);
    result.put("lgbmPrediction", lgbmPred);
    result.put("ensemblePrediction", ensemble);
    result.put("modelReady", ready);
    result.put("modelMape", lgbmModel.backtestMape());
    result.put("modelTrees", lgbmModel.treeCount());
    result.put("trainedRows", lgbmModel.trainedRows());
    result.put("recent7", recent7);
    result.put("histMean", Math.round(histMean * 10) / 10.0);
    result.put("histMin", histMin);
    result.put("histMax", histMax);
    result.put("decisionContributions", dual.getOrDefault("decisionContributions", List.of()));
    result.put("decisionRules", dual.getOrDefault("decisionRules", List.of()));
    List<Map<String, Object>> checks = new ArrayList<>();
    checks.add(check("模型运行时就绪", ready,
        ready ? lgbmModel.treeCount() + "棵回归树已加载, 纯Java推理与训练侧预测零误差" : "模型未就绪, 当前
        double mapeNum;
        try { mapeNum = Double.parseDouble(lgbmModel.backtestMape()); }
        catch (NumberFormatException e) { mapeNum = 999; }
        checks.add(check("回测精度可信", mapeNum < 10,
    
```

```

        "university_food_waste " + lgbmModel.trainedRows() + "行真实数据滚动回测 MAPE=" + lgbmModel.backtestMap
    if (lgbmPred != null) {
        double divergence = Math.abs(lgbmPred - rulePred) * 100.0 / Math.max(1, rulePred);
        checks.add(check("训练模型与规则引擎分歧度", divergence <= 30,
            "LGBM=" + lgbmPred + "份 vs 规则=" + rulePred + "份, 分歧 " + String.format("%.1f", divergence)
        boolean inHist = histMax == 0 || (lgbmPred >= histMin * 0.6 && lgbmPred <= histMax * 1.4);
        checks.add(check("预测落在历史合理区间", inHist,
            histMax == 0 ? "历史数据不足, 跳过区间校验"
                : "近7日销量区间 " + histMin + "-" + histMax + " 份, 模型预测 " + lgbmPred + " 份"));
    }
    long passCount = checks.stream().filter(c -> Boolean.TRUE.equals(c.get("pass"))).count();
    String detVerdict = passCount == checks.size() ? "全部通过" : (passCount >= checks.size() - 1 ? "基本通过" : "
    Map<String, Object> deterministic = new LinkedHashMap<>();
    deterministic.put("verdict", detVerdict);
    deterministic.put("passCount", passCount);
    deterministic.put("totalCount", checks.size());
    deterministic.put("checks", checks);
    result.put("deterministic", deterministic);
    List<Integer> chronological = new ArrayList<>(recent7);
    Collections.reverse(chronological);
    result.put("llm", runLlmCheck(weather, examWeek, campusEvent,
        rulePred, lgbmPred, histMean, histMin, histMax, chronological, dual));
    return result;
}

private Map<String, Object> runLlmCheck(String weather, boolean examWeek, boolean campusEvent,
    int rulePred, Integer lgbmPred,
    double histMean, int histMin, int histMax,
    List<Integer> recent7Chronological, Map<String, Object> dual) {
    Map<String, Object> llm = new LinkedHashMap<>();
    llm.put("enabled", zhipuAiService.available());
    llm.put("model", zhipuAiService.modelName());
    if (!zhipuAiService.available()) {
        llm.put("status", "未配置大模型Key (foodwise.ai.api-key), 当前展示确定性交叉校验; 配置Key后此处显示大模
        return llm;
    }
    try {
        List<Map<String, Object>> topFactors = new ArrayList<>();
        @SuppressWarnings("unchecked")
        List<Map<String, Object>> contribs = (List<Map<String, Object>>) dual.getDefault("decisionContributions",
        for (Map<String, Object> c : contribs.subList(0, Math.min(5, contribs.size()))) {
            Map<String, Object> f = new LinkedHashMap<>();
            f.put("因子", c.get("featureZh"));
            f.put("对预测的影响_份", c.get("soldDelta"));
            f.put("命中分裂次数", c.get("splits"));
            topFactors.add(f);
        }
        Map<String, Object> facts = new LinkedHashMap<>();
        facts.put("情景", "高校食堂某菜品明日午餐销量独立预估");
        facts.put("菜品", String.valueOf(dual.get("dishName")));
        facts.put("明日星期", weekdayZh(LocalDate.now().plusDays(1)));
    }
}

```

```

facts.put("天气", weather);
facts.put("考试周", examWeek ? "是" : "否");
facts.put("校园活动", campusEvent ? "是" : "否");
facts.put("近7日实际销量_份_按日期从早到晚", recent7Chronological);
facts.put("近7日均值", histMean);
facts.put("近7日最小", histMin);
facts.put("近7日最大", histMax);
facts.put("规则引擎预测_份", rulePred);
facts.put("机器学习模型LGBM预测_份", lgbmPred);
facts.put("LGBM训练依据Top因子", topFactors);
facts.put("LGBM回测MAPE百分比", lgbmModel.backtestMape());
String system = """
    你是高校食堂需求评估专家。用户会给出同一菜品的真实历史销量、明日情景、规则引擎预测和机器学习模型LG
    你必须独立判断，不要迎合任何一方的数字。
    只返回JSON对象：{"llmLow":整数,"llmHigh":整数,"verdict":"agree或caution或disagree","comment":"60
    规则：llmLow/llmHigh是你独立预估的明日销量区间；verdict表示你是否认可LGBM预测（agree认可/caution
    comment只陈述你的依据，不得编造事实中没有的数据。
    """;

Optional<Map<String, Object>> resp = zhipuAiService.generateJson(system, facts);
if (resp.isEmpty()) {
    llm.put("status", "大模型服务调用失败，本次仅展示确定性交叉校验");
    return llm;
}
Map<String, Object> r = resp.get();
int llmLow = asInt(r.get("llmLow"));
int llmHigh = asInt(r.get("llmHigh"));
String verdict = String.valueOf(r.getOrDefault("verdict", "")).toLowerCase();
String comment = String.valueOf(r.getOrDefault("comment", ""));
llm.put("llmLow", llmLow);
llm.put("llmHigh", llmHigh);
llm.put("verdict", verdict);
llm.put("comment", comment);
if (lgbmPred != null && llmHigh >= llmLow && llmLow > 0) {
    String consistency;
    if (lgbmPred >= llmLow && lgbmPred <= llmHigh) consistency = "一致";
    else if (lgbmPred >= llmLow * 0.9 && lgbmPred <= llmHigh * 1.1) consistency = "接近";
    else consistency = "分歧";
    llm.put("consistency", consistency);
} else {
    llm.put("consistency", "无法判定");
}
llm.put("status", "ok");
return llm;
} catch (Exception e) {
    llm.put("status", "大模型混测异常：" + e.getMessage());
    return llm;
}
}

private Map<String, Object> check(String name, boolean pass, String detail) {
    Map<String, Object> m = new LinkedHashMap<>();

```

```

        m.put("name", name);
        m.put("pass", pass);
        m.put("detail", detail);
        return m;
    }

    private int intVal(Object o) {
        return o instanceof Number n ? n.intValue() : 0;
    }

    private int asInt(Object o) {
        if (o instanceof Number n) return n.intValue();
        try { return (int) Double.parseDouble(String.valueOf(o).replaceAll("[^0-9.\\-]", "")); }
        catch (Exception e) { return 0; }
    }

    private String weekdayZh(LocalDate date) {
        return List.of("周一", "周二", "周三", "周四", "周五", "周六", "周日").get(date.getDayOfWeek().getValue() - 1);
    }
}

// ===== org/foodwise/campus/service/IntelligentDecisionService.java =====
package org.foodwise.campus.service;
import tools.jackson.core.JsonException;
import tools.jackson.databind.ObjectMapper;
import org.foodwise.campus.model.IntelligentAdvice;
import org.foodwise.campus.model.OfferRecommendation;
import org.foodwise.campus.repository.FoodwiseRepository;
import org.springframework.stereotype.Service;
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
@Service
public class IntelligentDecisionService {
    private final FoodwiseRepository repository;
    private final ZhipuAiService zhipuAiService;
    private final ObjectMapper objectMapper;
    public IntelligentDecisionService(FoodwiseRepository repository, ZhipuAiService zhipuAiService,
                                      ObjectMapper objectMapper) {
        this.repository = repository;
        this.zhipuAiService = zhipuAiService;
        this.objectMapper = objectMapper;
    }

    public IntelligentAdvice dashboardAdvice() {
        Map<String, Object> summary = repository.dashboardSummary();
        List<Map<String, Object>> risks = repository.riskDishes();
        double prepared = number(summary.get("prepared"));
        double sold = number(summary.get("sold"));
        double leftover = number(summary.get("leftover"));
    }
}

```

```

double leftoverRate = prepared == 0 ? 0 : leftover * 100 / prepared;
long mediumHigh = risks.stream().filter(row -> !"低".equals(String.valueOf(row.get("risk_level")))).count();
Map<String, Object> facts = new LinkedHashMap<>();
facts.put("prepared", prepared);
facts.put("sold", sold);
facts.put("leftover", leftover);
facts.put("leftoverRate", round(leftoverRate));
facts.put("rescued", number(summary.get("rescued")));
facts.put("mediumHighRiskDishes", mediumHigh);
facts.put("topRiskDish", risks.isEmpty() ? "暂无" : risks.get(0).get("name"));
long highRisk = risks.stream().filter(row -> "高".equals(String.valueOf(row.get("risk_level")))).count();
String riskLevel = leftoverRate >= 10 || highRisk >= 2 ? "HIGH" : leftoverRate >= 6 || mediumHigh >= 2 ? "MEDI
String localSummary = leftoverRate >= 8
    ? "最近营业日剩余压力偏高，应优先控制高风险菜品首批备餐，并延后非必要补餐。"
    : "最近营业日供需整体稳定，可继续保持分批备餐，并重点跟踪少数高风险菜品。";
List<String> reasons = new ArrayList<>();
reasons.add("备餐" + integer(prepared) + "份，售出" + integer(sold) + "份，剩余率" + round(leftoverRate) + "
reasons.add("当前共有" + mediumHigh + "个中高风险菜品");
if (!risks.isEmpty()) reasons.add("剩余风险最高的是" + risks.get(0).get("name"));
List<String> actions = List.of("先处理高风险菜品", "按销量节点决定是否补餐", "闭餐后记录采纳结果");
return createAdvice("今日经营简报", "dashboard", facts, localSummary, riskLevel, reasons, actions,
    List.of("建议仅用于经营辅助，食品安全与现场动作由档口负责人确认"));
}

public IntelligentAdvice predictionAdvice(String dishName, int low, int mid, int high, int firstBatch,
    int replenish, int stopHour, int stopMinute, BigDecimal confidence,
    String weather, boolean examWeek, boolean campusEvent,
    List<String> factorDetails) {
    Map<String, Object> facts = new LinkedHashMap<>();
    facts.put("dishName", dishName);
    facts.put("predictedRange", List.of(low, high));
    facts.put("predictedMid", mid);
    facts.put("firstBatch", firstBatch);
    facts.put("replenishQty", replenish);
    facts.put("stopTime", "%02d:%02d".formatted(stopHour, stopMinute));
    facts.put("confidence", confidence);
    facts.put("weather", weather);
    facts.put("examWeek", examWeek);
    facts.put("campusEvent", campusEvent);
    facts.put("factors", factorDetails);
    String risk = confidence.doubleValue() < 78 ? "MEDIUM" : "LOW";
    String localSummary = dishName + "预计需求为" + low + "—" + high + "份，建议先准备" + firstBatch
        + "份，再根据实际销售决定是否补充" + replenish + "份。";
    List<String> reasons = new ArrayList<>(factorDetails);
    List<String> actions = List.of(
        "首批备餐控制在" + firstBatch + "份",
        "销售达到计划节点后再补充" + replenish + "份",
        "%02d:%02d后原则上停止继续制作".formatted(stopHour, stopMinute)
    );
    return createAdvice("备餐决策说明", "prediction", facts, localSummary, risk, reasons, actions,
        List.of("需求区间不是销量承诺，异常天气或活动需人工确认"));
}

```

```

}

public IntelligentAdvice reportAdvice() {
    Map<String, Object> report = repository.reportSummary();
    Map<String, Object> facts = new LinkedHashMap<>(report);
    double improvement = number(report.get("baseline_leftover_rate")) - number(report.get("intervention_leftover_
String localSummary = "经营测算中, 剩余率较基准阶段改善" + round(improvement)
    + "个百分点。下一周期应继续记录采纳、交付工时和增量毛利。";
    List<String> reasons = List.of(
        "剩余率由" + report.get("baseline_leftover_rate") + "%变为" + report.get("intervention_leftover_rate"
        "策略阶段建议执行率为" + report.get("adoption_rate") + "%",
        "优惠销售共记录" + report.get("rescued_meals") + "份"
    );
    return createAdvice("周期经营复盘", "report", facts, localSummary, "LOW", reasons,
        List.of("保留有效备餐参数", "复查未采纳建议原因", "用真实付费和交付工时校准商业价值"),
        List.of("当前结果属于经营测算, 不替代真实档口连续台账"));
}

public OfferRecommendation recommendOffer(long dishId) {
    Map<String, Object> dish = repository.dish(dishId);
    Map<String, Object> operation = repository.latestDishOperation(dishId);
    int leftover = integer(number(operation.get("leftover_qty")));
    int sold = integer(number(operation.get("sold_qty")));
    int prepared = integer(number(operation.get("prepared_qty")));
    BigDecimal price = decimal(dish.get("price"));
    BigDecimal unitCost = decimal(dish.get("unit_cost"));
    int qty = Math.max(1, Math.min(leftover, Math.max(3, (int) Math.ceil(prepared * 0.08))));
    int discountRate = leftover >= 7 ? 20 : leftover >= 4 ? 15 : 10;
    BigDecimal offerPrice = price.multiply(BigDecimal.valueOf(100 - discountRate))
        .divide(BigDecimal.valueOf(100), 1, RoundingMode.HALF_UP)
        .max(unitCost.multiply(BigDecimal.valueOf(1.15)).setScale(1, RoundingMode.HALF_UP));
    int recovered = Math.max(1, (int) Math.round(qty * (discountRate >= 20 ? 0.8 : 0.65)));
    LocalTime start = LocalTime.of(12, 20);
    LocalTime end = LocalTime.of(12, 45);
    Map<String, Object> facts = Map.of(
        "dishName", dish.get("name"), "prepared", prepared, "sold", sold, "leftover", leftover,
        "originalPrice", price, "recommendedPrice", offerPrice, "recommendedQty", qty,
        "startTime", start.toString(), "endTime", end.toString()
    );
    IntelligentAdvice advice = createAdvice("限时优惠建议", "offer", facts,
        "当前预计剩余" + leftover + "份, 建议最多发布" + qty + "份限时优惠, 并由档口确认餐品状态和截止时间。",
        leftover >= 7 ? "HIGH" : leftover >= 4 ? "MEDIUM" : "LOW",
        List.of("备餐" + prepared + "份, 已售" + sold + "份", "优惠数量不超过当前预计剩余", "建议价格高于单位成
        List.of("确认餐品状态", "按建议数量限量发布", "到达截止时间自动下架"),
        List.of("系统不判断餐品是否安全, 发布前必须人工确认"));
    return new OfferRecommendation(dishId, String.valueOf(dish.get("name")), qty, offerPrice, discountRate,
        start.format(DateTimeFormatter.ofPattern("HH:mm")), end.format(DateTimeFormatter.ofPattern("HH:mm")),
        recovered, offerPrice.multiply(BigDecimal.valueOf(recovered)), advice.riskLevel(), advice);
}

public void saveFeedback(long adviceId, boolean adopted, String adjustedValue, String rejectReason) {
    repository.saveAdviceFeedback(adviceId, adopted, adjustedValue, rejectReason);
}
}

```

```

private IntelligentAdvice createAdvice(String title, String scenario, Map<String, Object> facts,
                                       String localSummary, String riskLevel, List<String> localReasons,
                                       List<String> localActions, List<String> localWarnings) {
    ZhipuAiService.AiNarrative narrative = zhipuAiService.generate(scenario, facts).orElse(null);
    boolean aiGenerated = narrative != null && !narrative.summary().isBlank();
    String summary = aiGenerated ? narrative.summary() : localSummary;
    List<String> reasons = localReasons;
    List<String> actionTexts = localActions;
    List<String> warnings = localWarnings;
    List<IntelligentAdvice.Action> actions = new ArrayList<>();
    for (int index = 0; index < actionTexts.size(); index++) {
        actions.add(new IntelligentAdvice.Action("ACTION_" + (index + 1), "建议" + (index + 1),
            actionTexts.get(index), index == 0 ? "HIGH" : "MEDIUM"));
    }
    String source = aiGenerated ? "经营台账 • 规则校验 • 经营概括" : "经营台账 • 规则判断";
    long id = repository.saveAdvice(scenario, title, summary, riskLevel, source, aiGenerated,
        json(facts), json(Map.of("reasons", reasons, "actions", actions, "warnings", warnings)));
    return new IntelligentAdvice(id, title, summary, riskLevel, reasons, actions, warnings, source,
        aiGenerated, LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm")));
}

private String json(Object value) {
    try {
        return objectMapper.writeValueAsString(value);
    } catch (JacksonException exception) {
        return "{}";
    }
}

private double number(Object value) {
    return value instanceof Number number ? number.doubleValue() : value == null ? 0 : Double.parseDouble(String.
}

private int integer(double value) {
    return (int) Math.round(value);
}

private double round(double value) {
    return Math.round(value * 10.0) / 10.0;
}

private BigDecimal decimal(Object value) {
    return value instanceof BigDecimal decimal ? decimal : new BigDecimal(String.valueOf(value));
}

}

// ===== org/foodwise/campus/service/OperationFeedbackService.java =====
package org.foodwise.campus.service;
import org.foodwise.campus.model.OperationLearningResult;
import org.foodwise.campus.repository.FoodwiseRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.LocalDate;
import java.util.HashMap;
import java.util.List;

```

```

import java.util.Map;
@Service
public class OperationFeedbackService {
    private static final String LEARNING_METHOD = "最近7日递增权重滚动更新";
    private final FoodwiseRepository repository;
    private final PredictionService predictionService;
    public OperationFeedbackService(FoodwiseRepository repository, PredictionService predictionService) {
        this.repository = repository;
        this.predictionService = predictionService;
    }
    @Transactional
    public OperationLearningResult submit(LocalDate businessDate, long dishId, int plannedQty,
                                          int preparedQty, int soldQty, int discountSoldQty,
                                          int leftoverQty, BigDecimal revenue, String weather,
                                          String eventTag, boolean recommendationAdopted,
                                          boolean safetyConfirmed, String operatorName, String note) {
        if (businessDate == null || businessDate.isAfter(LocalDate.now())) {
            throw new IllegalArgumentException("营业日期不能为空，也不能晚于今天");
        }
        if (!safetyConfirmed) {
            throw new IllegalArgumentException("请先确认回传数据对应当餐经营记录和食品安全边界");
        }
        if (preparedQty != soldQty + leftoverQty) {
            throw new IllegalArgumentException("数量不守恒：备餐量应等于售出量与剩余量之和");
        }
        if (discountSoldQty > soldQty) {
            throw new IllegalArgumentException("优惠售出量不能大于总售出量");
        }
        if (plannedQty < 0 || preparedQty < 0 || soldQty < 0 || discountSoldQty < 0 || leftoverQty < 0) {
            throw new IllegalArgumentException("经营数量不能为负数");
        }
        if (revenue == null || revenue.signum() < 0) {
            throw new IllegalArgumentException("营业收入不能为空或为负数");
        }
        Map<String, Object> dish = repository.dish(dishId);
        List<Integer> beforeSales = repository.recentSales(dishId);
        BigDecimal baselineBefore = weightedAverage(beforeSales);
        FoodwiseRepository.OperationSaveResult saved = repository.upsertOperation(
            businessDate, dishId, plannedQty, preparedQty, soldQty, discountSoldQty,
            leftoverQty, revenue.setScale(2, RoundingMode.HALF_UP), weather,
            eventTag, recommendationAdopted);
        repository.saveOperationFeedback(saved.operationId(), operatorName, safetyConfirmed, note,
            saved.updated() ? "修正" : "新增");
        List<Integer> afterSales = repository.recentSales(dishId);
        BigDecimal baselineAfter = weightedAverage(afterSales);
        BigDecimal adjustmentRate = baselineBefore.signum() == 0 ? BigDecimal.ZERO : baselineAfter
            .subtract(baselineBefore).multiply(BigDecimal.valueOf(100))
            .divide(baselineBefore, 2, RoundingMode.HALF_UP);
        repository.saveLearningLog(saved.operationId(), dishId, LEARNING_METHOD, afterSales.size(),
            baselineBefore, baselineAfter, adjustmentRate);
    }
}

```

```

repository.replaceOperationAlerts(businessDate, dishId, preparedQty, soldQty, leftoverQty);
BigDecimal leftoverRate = preparedQty == 0 ? BigDecimal.ZERO : BigDecimal.valueOf(leftoverQty * 100.0 / prep
    .setScale(1, RoundingMode.HALF_UP);
String direction = adjustmentRate.signum() > 0 ? "上调" : adjustmentRate.signum() < 0 ? "下调" : "保持";
String summary = String.format("%s的经营记录已%s。纳入本次销量后, 近7日加权需求基线由%s份%s为%s份。",
    dish.get("name"), saved.updated() ? "修正" : "写入", baselineBefore, direction, baselineAfter);
String nextAction = leftoverRate.compareTo(BigDecimal.valueOf(8)) >= 0
    ? "剩余率偏高, 下一次备餐建议将优先控制首批投入, 并保留人工调整。"
    : "供需处于可控范围, 新记录将在下一次备餐预测和经营复盘直接使用。";
autoUpdateModel(dishId);
return new OperationLearningResult(saved.operationId(), dishId, String.valueOf(dish.get("name")),
    businessDate, saved.updated(), afterSales.size(), baselineBefore, baselineAfter,
    adjustmentRate, leftoverRate, LEARNING_METHOD, summary, nextAction);
}

private void autoUpdateModel(long dishId) {
    try {
        long newFeedbackCount = repository.countFeedbackSinceLastUpdate(dishId);
        if (newFeedbackCount >= 4) {
            List<Integer> sales = repository.recentSales(dishId);
            if (sales.size() < 4) {
                return; // 数据不足, 跳过更新
            }
            double newWeatherFactor = recalculateWeatherFactor(dishId);
            double newTrendFactor = recalculateTrendFactor(sales);
            Map<String, Double> factors = new HashMap<>();
            factors.put("weatherFactor", newWeatherFactor);
            predictionService.updateFactors(dishId, factors);
            predictionService.updateTrendFactor(dishId, newTrendFactor);
            System.out.printf("反馈闭环自动更新: 菜品%d 天气因子调整为 %.2f, 趋势因子调整为 %.2f (基于%d条新反馈)
                dishId, newWeatherFactor, newTrendFactor, newFeedbackCount);
        }
    } catch (Exception e) {
        System.err.println("反馈闭环自动更新失败 (菜品" + dishId + ") : " + e.getMessage());
    }
}

private double recalculateWeatherFactor(long dishId) {
    try {
        List<Map<String, Object>> recentOps = repository.recentOperationsByDish(dishId);
        if (recentOps.isEmpty()) {
            return 1.0;
        }
        Map<String, Integer> weatherCount = new HashMap<>();
        for (Map<String, Object> op : recentOps) {
            String w = String.valueOf(op.get("weather"));
            weatherCount.merge(w, 1, Integer::sum);
        }
        String mostCommonWeather = weatherCount.entrySet().stream()
            .max(Map.Entry.comparingByValue())
            .map(Map.Entry::getKey)
            .orElse("晴朗");
    }
}

```

```

        double wc = predictionService.weatherCode(mostCommonWeather);
        return switch (mostCommonWeather) {
            case "小雨" -> 0.92;
            case "大雨" -> 0.82;
            case "高温" -> 0.90;
            case "降温" -> 1.04;
            default -> 1.00;
        };
    } catch (Exception e) {
        return 1.0;
    }
}

private double recalculateTrendFactor(List<Integer> sales) {
    if (sales.size() < 4) return 1.0;
    List<Integer> chronological = new java.util.ArrayList<>(sales);
    java.util.Collections.reverse(chronological);
    double early = chronological.subList(0, chronological.size() / 2).stream()
        .mapToInt(Integer::intValue).average().orElse(1);
    double recent = chronological.subList(chronological.size() / 2, chronological.size()).stream()
        .mapToInt(Integer::intValue).average().orElse(early);
    return Math.max(0.92, Math.min(1.08, recent / early));
}

public Map<String, Object> learningStatus() {
    return repository.learningStatus();
}

public List<Map<String, Object>> recentFeedback() {
    return repository.recentOperationFeedback();
}

private BigDecimal weightedAverage(List<Integer> newestFirst) {
    if (newestFirst.isEmpty()) return BigDecimal.ZERO.setScale(1);
    double total = 0;
    double weights = 0;
    for (int index = 0; index < newestFirst.size(); index++) {
        double weight = newestFirst.size() - index;
        total += newestFirst.get(index) * weight;
        weights += weight;
    }
    return BigDecimal.valueOf(total / weights).setScale(1, RoundingMode.HALF_UP);
}
}

// ===== org/foodwise/campus/service/ZhipuAiService.java =====
package org.foodwise.campus.service;
import tools.jackson.databind.JsonNode;
import tools.jackson.databind.ObjectMapper;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;

```

```

import java.time.Duration;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.HashSet;
import java.util.Set;
import java.util.regex.Matcher;
import java.util.regex.Pattern;
@Service
public class ZhipuAiService {
    private final ObjectMapper objectMapper;
    private final HttpClient httpClient;
    private final String apiKey;
    private final String model;
    private final String endpoint;
    private final boolean enabled;
    public ZhipuAiService(
        ObjectMapper objectMapper,
        @Value("${foodwise.ai.api-key:}") String apiKey,
        @Value("${foodwise.ai.model:glm-4-flash}") String model,
        @Value("${foodwise.ai.endpoint:https://open.bigmodel.cn/api/paas/v4/chat/completions}") String endpoint,
        @Value("${foodwise.ai.enabled:true}") boolean enabled
    ) {
        this.objectMapper = objectMapper;
        this.apiKey = apiKey == null ? "" : apiKey.trim();
        this.model = model;
        this.endpoint = endpoint;
        this.enabled = enabled;
        this.httpClient = HttpClient.newBuilder()
            .connectTimeout(Duration.ofSeconds(4))
            .build();
    }
    public boolean available() {
        return enabled && !apiKey.isBlank();
    }
    public String modelName() {
        return model;
    }
    public Optional<AiNarrative> generate(String scenario, Map<String, Object> facts) {
        if (!available()) return Optional.empty();
        try {
            String system = """
                你只负责填写高校餐饮经营页面中的“经营概括”文字槽位，不负责计算、决策或改变展示结构。
                所有数量、金额、时间、比例、风险等级、原因、经营动作和食品安全提示均由后端固定模板提供。
                只返回JSON对象：{"summary":"简短概括"}。summary不超过70个汉字；如需引用数字，只能原样引用用户事
                不增加用户事实中没有的结论，不使用夸张宣传语言，不返回标题、列表、Markdown或额外字段。
                """;
            Map<String, Object> userPayload = Map.of("scenario", scenario, "facts", facts);

```

```

Map<String, Object> requestBody = new LinkedHashMap<>();
requestBody.put("model", model);
requestBody.put("temperature", 0.2);
requestBody.put("max_tokens", 650);
requestBody.put("response_format", Map.of("type", "json_object"));
requestBody.put("messages", List.of(
    Map.of("role", "system", "content", system),
    Map.of("role", "user", "content", objectMapper.writeValueAsString(userPayload))
));
HttpRequest request = HttpRequest.newBuilder(URI.create(endpoint))
    .timeout(Duration.ofSeconds(12))
    .header("Authorization", "Bearer " + apiKey)
    .header("Content-Type", "application/json; charset=UTF-8")
    .POST(HttpRequest.BodyPublishers.ofString(objectMapper.writeValueAsString(requestBody), StandardCharsets.UTF_8))
    .build();
HttpResponse<String> response = httpClient.send(request, HttpResponse.BodyHandlers.ofString(StandardCharsets.UTF_8));
if (response.statusCode() < 200 || response.statusCode() >= 300) return Optional.empty();
JsonNode root = objectMapper.readTree(response.body());
String content = root.path("choices").path(0).path("message").path("content").asText("");
JsonNode narrative = objectMapper.readTree(extractJson(content));
String summary = safeText(narrative.path("summary").asText(), 80);
if (!usesOnlyProvidedNumbers(summary, userPayload)) summary = "";
return Optional.of(new AiNarrative(
    summary,
    stringList(narrative.path("reasons"), 4),
    stringList(narrative.path("actions"), 4),
    stringList(narrative.path("warnings"), 3)
));
} catch (Exception ignored) {
    return Optional.empty();
}
}

@SuppressWarnings("unchecked")
public Optional<Map<String, Object>> generateJson(String systemPrompt, Map<String, Object> facts) {
    if (!available()) return Optional.empty();
    try {
        Map<String, Object> requestBody = new LinkedHashMap<>();
        requestBody.put("model", model);
        requestBody.put("temperature", 0.2);
        requestBody.put("max_tokens", 650);
        requestBody.put("response_format", Map.of("type", "json_object"));
        requestBody.put("messages", List.of(
            Map.of("role", "system", "content", systemPrompt),
            Map.of("role", "user", "content", objectMapper.writeValueAsString(facts))
        ));
        HttpRequest request = HttpRequest.newBuilder(URI.create(endpoint))
            .timeout(Duration.ofSeconds(15))
            .header("Authorization", "Bearer " + apiKey)
            .header("Content-Type", "application/json; charset=UTF-8")
            .POST(HttpRequest.BodyPublishers.ofString(objectMapper.writeValueAsString(requestBody), StandardCharsets.UTF_8))
    }
}

```

```

        .build();
        HttpResponse<String> response = httpClient.send(request, HttpResponse.BodyHandlers.ofString(StandardCharsets
        if (response.statusCode() < 200 || response.statusCode() >= 300) return Optional.empty();
        JsonNode root = objectMapper.readTree(response.body());
        String content = root.path("choices").path(0).path("message").path("content").asText("");
        JsonNode parsedNode = objectMapper.readTree(extractJson(content));
        Map<String, Object> parsed = objectMapper.convertValue(parsedNode, Map.class);
        return Optional.of(parsed);
    } catch (Exception ignored) {
        return Optional.empty();
    }
}

private String extractJson(String text) {
    int start = text.indexOf('{');
    int end = text.lastIndexOf('}');
    return start >= 0 && end > start ? text.substring(start, end + 1) : "{}";
}

private List<String> stringList(JsonNode node, int limit) {
    List<String> values = new ArrayList<>();
    if (!node.isArray()) return values;
    for (JsonNode item : node) {
        String value = safeText(item.asText(), 70);
        if (!value.isBlank()) values.add(value);
        if (values.size() >= limit) break;
    }
    return values;
}

private String safeText(String value, int maxLength) {
    if (value == null) return "";
    String cleaned = value.replaceAll("[\\r\\n]+", " ").trim();
    return cleaned.length() <= maxLength ? cleaned : cleaned.substring(0, maxLength);
}

private boolean usesOnlyProvidedNumbers(String summary, Map<String, Object> userPayload) {
    Pattern numberPattern = Pattern.compile("\\d+(?:\\.\\d+)?");
    Set<String> allowed = new HashSet<>();
    try {
        Matcher factNumbers = numberPattern.matcher(objectMapper.writeValueAsString(userPayload));
        while (factNumbers.find()) allowed.add(factNumbers.group());
    } catch (Exception exception) {
        return false;
    }
    Matcher summaryNumbers = numberPattern.matcher(summary);
    while (summaryNumbers.find()) {
        if (!allowed.contains(summaryNumbers.group())) return false;
    }
    return true;
}

public record AiNarrative(String summary, List<String> reasons, List<String> actions, List<String> warnings) {
}
}

```